VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

MATHEMATICAL MODELING (CO2011)

Assignment

"Cutting stock problem"

Instructor(s): Nguyễn Văn Minh Mẫn, *Mahidol University*
Nguyễn An Khương, *CSE-HCMUT*
Lê Hồng Trang, *CSE-HCMUT*
Mai Xuân Toàn, *CSE-HCMUT*
Trần Hồng Tài, *CSE-HCMUT*
Nguyễn Quang Đức, *CSE-HCMUT*

Students: Nguyễn Văn Đức - 2310790 *(Group L05 - Team 43, **Leader**)*
Phan Thị Cẩm Vân - 2313873 *(Group L05 - Team 43)*
Nguyễn Tuấn Hiệp - 2311011 *(Group L06 - Team 43)*
Mau Gia Kiệt - 2311770 *(Group L05 - Team 43)*
Trương Nguyễn Gia Bảo - 2310271 *(Group L05 - Team 43)*

HO CHI MINH CITY, AUGUST 2024

Contents

List of Symbols **3**

List of Acronyms **3**

List of Figures **5**

List of Tables **5**

Member list & Workload **5**

1 Giới thiệu **6**
1.1 Tổng quan về Cutting Stock Problem (CSP) . . . 6
1.1.1 Lịch sử hình thành . . . 6
1.1.2 Phân loại CSP theo số chiều không gian . . . 6
1.2 Bài toán 2D CSP . . . 7

2 Lời cảm ơn **9**

3 Mô hình hoá bài toán theo quy hoạch tuyến tính nguyên **10**
3.1 Mô hình hóa bài toán trong trường hợp mẫu vật liệu có kích thước cố định . . . 10
3.2 Mô hình hóa bài toán trong trường hợp mẫu vật liệu có nhiều kích thước khác nhau với số lần sử dụng không giới hạn . . . 15
3.3 Mô hình hóa bài toán trong trường hợp chỉ có hữu hạn các mẫu vật liệu với kích thước khác nhau . . . 16

4 Các giải thuật **20**
4.1 Giải thuật Column Generation . . . 20
4.1.1 Ý tưởng . . . 20
4.1.2 Mô hình hóa bài toán phụ của giải thuật Column Generation . . . 22
4.1.3 Mã giả . . . 25
4.1.4 Ưu và nhược điểm của giải thuật Column Generation . . . 25
4.2 Giải thuật tham lam kết hợp Simulated Annealing . . . 26
4.2.1 Giải thuật tham lam (Greedy Algorithm) . . . 26
4.2.2 Giải thuật Simulated Annealing (SA) . . . 27
4.2.3 Ưu và nhược điểm của giải thuật tham lam kết hợp Simulated Annealing 30
4.3 Hướng tiếp cận bằng Reinforcement Learning . . . 32
4.3.1 Sơ lược về Policy Gradient . . . 32
4.3.2 Mô tả State Space, Action Space và phần thưởng . . . 34
4.3.3 Entropy Regularization . . . 36
4.3.4 Ưu và nhược điểm của Reinforcement Learning . . . 37

5 Đánh giá hiệu suất **39**
5.1 Đánh giá hiệu suất của phương pháp Simplex . . . 39
5.2 Đánh giá hiệu suất của giải thuật Column Generation . . . 41
5.3 Đánh giá hiệu suất của giải thuật tham lam kết hợp Simulated Annealing . . . 41
5.4 Đánh giá hiệu suất của mô hình Reinforcement Learning . . . 43
5.4.1 Tổng quan cấu trúc mạng thần kinh . . . 43
5.4.2 Đánh giá hiệu suất với một bộ dữ liệu . . . 44

5.5 Case Study: Ứng dụng 2D CSP trong ngành sản xuất bao bì 45

6 Kết luận **54**
6.1 Hướng dẫn sử dụng công cụ 54
6.2 Kết quả làm việc 55
6.3 Các thiếu sót 55
6.4 Hướng phát triển trong tương lai 55

List of Symbols

$\mathbb{N}$ Set of natural numbers

$\mathbb{R}$ Set of real numbers

$\mathbb{R}^+$ Set of positive real numbers

List of Acronyms

CSP Cutting Stock Problem

SA Simulated Annealing

RL Reinforcement Learning

DCG Delayed Column Generation

Fig. Figure

Tab. Table

Sys. System of Equations

Eq. Equation

e.g. For Example

i.e. That Is

List of Figures

1 *(a) Kiểu cắt 2-Stage Guillotine không xén. (b) Kiểu cắt 2-Stage Guillotine có xén.* Source: Andrade Birgin Morabito . . . 7
2 *Minh họa các mẫu vật liệu chồng lên nhau (Item 2 và 4 bị quay 90 độ).* . . . 11
3 *Minh họa cách giảm số lượng biến nhị phân khi xuất hiện ràng buộc SOS1.* . . . 14
4 *Minh họa cách xác định tập* A_3. . . . 14
5 *Minh họa cách xác định* b_i. . . . 16
6 *(a) Mẫu vật liệu ban đầu. (b) Sau giai đoạn cắt đầu tiên. (c) Sau giai đoạn cắt thứ hai. (d) Sau khi cắt xén và loại bỏ vật liệu thừa.* . . . 20
7 *Minh họa các dải (Stripe) trong kiểu cắt 2-Stage Guillotine có xén.* . . . 23
8 *Minh họa cơ chế chọn dải để đưa vào mẫu vật liệu.* . . . 24
9 *Thứ tự cắt các sản phẩm theo giải thuật tham lam (sản phẩm có diện tích lớn nhất cắt trước, cắt theo quy tắc Bottom-Left).* . . . 27
10 *Minh họa phần diện tích bị đè lên nhau của các sản phẩm (phần gạch sọc).* . . . 29
11 *Mô tả các thành phần của State Vector.* . . . 35
12 *Minh họa cách cắt các tấm bìa Carton bằng phần mềm CutLogic 2D.* . . . 48
13 *Minh họa cách cắt các tấm bìa Carton theo giải thuật tham lam kết hợp Simulated Annealing (ảnh lấy từ phần mềm nhóm tạo).* . . . 50
14 *Minh họa cách cắt các tấm bìa Carton theo mô hình Reinforcement Learning (ảnh lấy từ phần mềm nhóm tạo).* . . . 52
15 *Giao diện phần mềm.* . . . 54

List of Tables

1 Member list & workload . . . 5
2 Mô tả vị trí tương đối của 2 sản phẩm. . . . 12
3 Kết quả đánh giá tốc độ của mô hình 1. . . . 39
4 Kết quả đánh giá tốc độ của mô hình 2. . . . 40
5 Kết quả đánh giá tốc độ của mô hình 3. . . . 40
6 Kết quả đánh giá tốc độ của giải thuật Column Generation. . . . 41
7 Kết quả đánh giá nội bộ của giải thuật tham lam kết hợp Simulated Annealing. . . . 42
8 Cấu trúc Stock Prediction Layers. . . . 43
9 Cấu trúc Product Prediction Layers. . . . 44
10 Cấu trúc Rotation Prediction Layers. . . . 44
11 Kết quả đánh giá nội bộ của mô hình Reinforcement Learning. . . . 45
12 Danh sách kích thước của 100 tấm bìa Carton. . . . 46
13 Danh sách các sản phẩm cần cắt. . . . 46
14 Cách cắt theo phần mềm CutLogic 2D. . . . 47
15 Cách cắt theo giải thuật tham lam kết hợp Simulated Annealing. . . . 49
16 Cách cắt theo mô hình Reinforcement Learning. . . . 51
17 So sánh hiệu suất của giải thuật tham lam kết hợp Simulated Annealing và mô hình Reinforcement Learning với phần mềm CutLogic 2D. . . . 53

Member list & Workload

No.	Fullname	Student ID	Problems	% done
1	Nguyễn Văn Đức	2310790	- Mô hình hóa bài toán - Tìm hiểu và hiện thực giải thuật Reinforcement Learning - Xây dựng công cụ để giải bài toán - Tổng hợp và viết lại báo cáo	100%
2	Phan Thị Cẩm Vân	2313873	- Tìm hiểu và hiện thực giải thuật Reinforcement Learning - Xây dựng công cụ để giải bài toán - Tìm kiếm các công cụ đã có để so sánh hiệu suất	100%
3	Nguyễn Tuấn Hiệp	2311011	- Tìm hiểu và hiện thực giải thuật Column Generation - Xây dựng công cụ để giải bài toán - Tìm kiếm các Case Study và Dataset	100%
4	Mau Gia Kiệt	2311770	- Tìm hiểu và hiện thực một vài giải thuật Heuristic - Xây dựng công cụ để giải bài toán - Tìm kiếm các Case Study và Dataset	100%
5	Trương Nguyễn Gia Bảo	2310271	- Giới thiệu tổng quan về bài toán - Tìm kiếm các Case Study và Dataset - Xây dựng công cụ để giải bài toán	100%

Table 1: Member list & workload

1 Giới thiệu

1.1 Tổng quan về Cutting Stock Problem (CSP)

1.1.1 Lịch sử hình thành

Bài toán cắt vật liệu, hay còn gọi là Cutting Stock Problem (CSP), cho tới nay vẫn là một trong những bài toán tối ưu hóa tổ hợp (Combinatorial Optimization) thu hút được nhiều sự quan tâm nghiên cứu. Nó được nhà toán học Leonid Kantorovich đưa ra lần đầu tiên vào năm 1939 trong quá trình nghiên cứu các phương pháp toán học nhằm tối ưu hóa quy trình sản xuất. CSP tập trung vào việc xác định cách hiệu quả nhất để cắt các vật liệu lớn thành các kích thước nhỏ hơn để đáp ứng các nhu cầu cụ thể, đồng thời giảm thiểu chi phí phát sinh do vật liệu thừa. Bài toán này đã có tác động lớn đến nhiều lĩnh vực công nghiệp, bao gồm sản xuất thép, gỗ, kính và giấy, cũng như trong dịch vụ Logistics và vận tải hàng hóa.

Trong những thập kỷ tiếp theo, bài toán cắt vật liệu đã được nghiên cứu một cách sâu rộng bởi các nhà toán học và chuyên gia tối ưu hóa, đặc biệt thông qua các phương pháp quy hoạch tuyến tính. Một cột mốc quan trọng trong sự phát triển của CSP diễn ra vào năm 1961, khi P.C. Gilmore và R.E. Gomory đề xuất một phương pháp dựa trên quy hoạch tuyến tính để giải quyết bài toán này, mở ra những hướng nghiên cứu và ứng dụng đa dạng trong thực tiễn. Họ phát triển các thuật toán dựa trên phương pháp Simplex để tìm kiếm các cột tối ưu thông qua việc giải quyết các bài toán phụ, giúp giảm thiểu độ phức tạp tính toán cho bài toán ban đầu.

Xét về khía cạnh độ phức tạp tính toán, bài toán CSP được xếp loại là NP-hard và có thể được quy về bài toán Knapsack. CSP có thể được mô hình hóa dưới dạng quy hoạch tuyến tính nguyên - ILP (Integer Linear Programming) hoặc MILP (Mixed Integer Linear Programming) tùy vào ngữ cảnh bài toán.

1.1.2 Phân loại CSP theo số chiều không gian

a. CSP một chiều (1D CSP)

1D CSP là loại bài toán CSP đơn giản nhất, chỉ liên quan đến việc cắt các vật liệu một chiều, như các vật liệu dạng thanh hoặc cuộn dài. Bài toán 1D CSP thường gặp trong các ngành công nghiệp như sản xuất dây điện, các thanh kim loại hay ống nhựa, nơi các vật liệu cần được cắt thành các kích thước nhỏ hơn để phục vụ cho nhu cầu tiêu dùng và sản xuất. Việc cắt chính xác không chỉ giúp tối ưu hóa việc sử dụng nguyên liệu mà còn giảm phế liệu và hạ giá thành sản xuất.

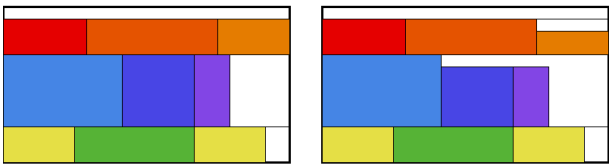
Vì sự đơn giản của bài toán, mà thuật toán chủ yếu dùng để tìm ra đáp án chính xác cho 1D CSP là Column Generation. Lý do chính cho việc này là việc xác định Pattern mới để thêm vào bài toán chính là đơn giản hơn nhiều so với trường hợp từ 2D trở lên. Ngoài ra, các thuật toán xấp xỉ cũng được áp dụng để tìm ra các kết quả gần đúng một cách nhanh chóng.

a. CSP hai chiều (2D CSP)

Trong các lĩnh vực như may mặc, sản xuất kính và gia công kim loại, bài toán cắt vật liệu hai chiều hay 2D CSP đã trở thành một vấn đề quan trọng nhằm tối ưu hóa việc sử dụng nguyên vật liệu. Giải quyết bài toán này sẽ giúp việc cắt các tấm vật liệu lớn thành các tấm nhỏ hơn không chỉ đáp ứng đủ nhu cầu về sản xuất mà còn giúp giảm thiểu lượng vật liệu thừa phát sinh. Để đạt được điều này, việc sắp xếp hợp lý các mẫu cắt trên bề mặt vật liệu hình chữ nhật là rất cần thiết, góp phần tối ưu hóa không gian sử dụng và hạ thấp chi phí sản xuất.

Hiện nay, bài toán 2D CSP có thể được tiếp cận bằng nhiều phương pháp và mô hình đa dạng tùy theo từng trường hợp cụ thể. Tiêu biểu như các mô hình liên quan đến việc cắt theo

kiểu Guillotine (Figure 1), với quá trình cắt được chia thành nhiều giai đoạn (Stage), mỗi giai đoạn ta có thể cắt nhiều lần, nhưng sẽ chỉ được cắt theo 1 hướng duy nhất, và hướng cắt của giai đoạn sau phải vuông góc với hướng cắt của giai đoạn ngay trước đó. Giải thuật Column Generation để giải bài toán này trong trường hợp cắt theo kiểu 2-Stage Guillotine có xén sẽ được trình bày tại subsection 4.1. Tuy nhiên, do sự phức tạp của bài toán, các thuật toán tối ưu hóa thuộc loại Heuristic thường được ưa chuộng hơn để tìm ra giải pháp gần đúng, như thuật toán di truyền, giải thuật tham lam, hay Reinforcement Learning, ... Bằng cách tối thiểu hóa số lượng tấm vật liệu sử dụng hoặc chi phí phát sinh do vật liệu thừa, 2D CSP không chỉ giúp giảm chi phí sản xuất mà còn nâng cao hiệu quả trong quy trình sản xuất của nhiều ngành công nghiệp khác nhau.



24×10 (a) 24×10 (b)

Figure 1: *(a) Kiểu cắt 2-Stage Guillotine không xén. (b) Kiểu cắt 2-Stage Guillotine có xén.* Source: Andrade Birgin Morabito

c. CSP ba chiều (3D CSP)

Bài toán 3D CSP cho đến nay vẫn chưa có nhiều ứng dụng đáng kể trong thực tiễn. Tuy nhiên các bài toán liên quan mật thiết tới nó như 3D Packing Problem lại được ứng dụng rộng rãi, đặc biệt là trong ngành Logistics và vận tải, khi việc sắp xếp hàng hóa để tối ưu hóa không gian và giảm chi phí là quan trọng hàng đầu. Cũng vì độ phức tạp cao, nên Reinforcement Learning thường là lựa chọn hợp lí nhất nếu ta muốn tìm ra giải pháp xấp xỉ cho 3D CSP trong một khoảng thời gian chấp nhận được.

1.2 Bài toán 2D CSP

Bài toán 2D CSP có mục tiêu là cắt ra các sản phẩm có dạng hình chữ nhật nhỏ từ các mẫu vật liệu (Stock) cũng có dạng hình chữ nhật nhưng lớn hơn. Mục đích của việc giải bài toán 2D CSP là để tìm ra cách cắt giúp tiết kiệm chi phí nhất mà vẫn đảm bảo cung cấp đủ số lượng sản phẩm được yêu cầu.

Mỗi cách cắt khác nhau trên một mẫu vật liệu được gọi là một Pattern (hay Configuration). Thông thường ta sẽ chỉ quan tâm đến số lượng các sản phẩm được cắt ra trong mỗi Pattern, thay vì vị trí được cắt của các sản phẩm đó. Như vậy mỗi Pattern sẽ ứng với một chi phí (Cost), chi phí đó tùy vào bài toán mà có thể được định nghĩa khác nhau, ví dụ ta có thể định nghĩa chi phí của một Pattern là diện tích còn thừa trong mẫu vật liệu sau khi cắt theo Pattern đó, hoặc cố định chi phí của Pattern bất kì là bằng 1, ...

Có nhiều biến thể khác nhau của bài toán 2D CSP, và trong bài báo cáo này, nhóm sẽ tập trung phân tích mô hình bài toán trong 3 trường hợp nổi bật. Trường hợp đầu tiên là khi kích thước các mẫu vật liệu là như nhau với chi phí của tất cả Pattern đều bằng 1, tức mục tiêu bài toán là giảm thiểu số lượng mẫu vật liệu sử dụng, tương đương giảm thiểu lượng vật liệu thừa.

Trường hợp thứ hai, là khi có nhiều loại vật liệu với kích thước khác nhau (Multiple Stock Size), và chi phí của mỗi Pattern sẽ bằng diện tích của mẫu vật liệu dùng để cắt Pattern đó, tức mục tiêu bài toán của ta là tối ưu tổng diện tích của các vật liệu được sử dụng sao cho nó nhỏ nhất, lưu ý ở đây số mẫu vật liệu được sử dụng là không giới hạn. Và trường hợp cuối cùng, cũng với nhiều loại vật liệu với kích thước khác nhau, nhưng số lượng mẫu vật liệu được dùng là có giới hạn, và chi phí của mỗi Pattern cũng sẽ bằng diện tích mẫu vật liệu chứa Pattern đó. Mô hình tương ứng của 3 trường hợp này sẽ được phân tích cụ thể ở section 3. Các giải thuật phù hợp sẽ được trình bày ở section 4.

2 Lời cảm ơn

Chúng em xin chân thành cảm ơn tất cả những người đã hỗ trợ nhóm trong suốt quá trình thực hiện dự án này. Trước hết, chúng em xin cảm ơn tất cả các thầy thuộc khoa Khoa học máy tính, vì sự hướng dẫn quý giá, những lời động viên và phản hồi sâu sắc của các thầy đã giúp chúng em rất nhiều trong việc hoàn thành dự án. Và đặc biệt, nhóm chúng em xin dành tặng lời tri ân tới thầy Lê Hồng Trang, người đã tận tình dẫn dắt chúng em trong môn Mô hình hóa toán học học kỳ 241. Sự hướng dẫn tận tâm và những kiến thức vô giá của thầy đã giúp nhóm chúng em có được những cái nhìn sâu sắc và toàn diện về quy hoạch tuyến tính nói chung và bài toán cắt vật liệu nói riêng, giúp chúng em thấy được ý nghĩa thực sự của nó trong đời sống thực tiễn.

Chúng em cũng xin chân thành cảm ơn sự nỗ lực của tất cả các thành viên trong nhóm đã góp phần vào sự thành công của bài báo cáo này. Những nỗ lực nghiên cứu sâu sắc về các thuật toán của các thành viên đã giúp cả nhóm hiểu rõ hơn về cách tiếp cận cũng như phương pháp giải quyết tối ưu cho bài toán 2D CSP. Bên cạnh đó, việc tìm hiểu lịch sử phát triển của bài toán 2D CSP đã mang đến cho chúng em cái nhìn tổng quan về bối cảnh và các tiến bộ trong lĩnh vực toán học nói chung và khoa học máy tính nói riêng. Với vai trò tổng hợp và viết lại báo cáo cuối cùng, em rất biết ơn sự hỗ trợ và cống hiến của tất cả các bạn. Sự hợp tác và sáng tạo của nhóm đã tạo nên một kết quả hoàn chỉnh và chất lượng. Xin chân thành cảm ơn!

3 Mô hình hoá bài toán theo quy hoạch tuyến tính nguyên

Tại section này, nhóm sẽ phân tích các mô hình của bài toán 2D CSP trong các điều kiện khác nhau. Đầu tiên là mô hình bài toán trong trường hợp mẫu vật liệu có kích thước cố định. Thứ hai là trong trường hợp mẫu vật liệu có nhiều kích thước khác nhau với mong muốn tối ưu tổng diện tích của các vật liệu sử dụng với số lần sử dụng là không giới hạn. Và cuối cùng cũng là với nhiều mẫu vật liệu có kích thước khác nhau, nhưng số lượng sử dụng là có giới hạn.

3.1 Mô hình hóa bài toán trong trường hợp mẫu vật liệu có kích thước cố định

Nhắc lại với trường hợp mẫu vật liệu có kích thước cố định, thì các Pattern sẽ có chi phí bằng 1, vì khi này tối thiểu hóa tổng số lượng mẫu vật liệu sử dụng cũng tương đương tối thiểu hóa tổng diện tích thừa. Giả sử ta có tối đa m mẫu vật liệu, với chiều rộng và chiều cao lần lượt là W và H, ta muốn cắt ra đúng n sản phẩm từ các mẫu vật liệu này, mỗi sản phẩm có chiều rộng và chiều cao lần lượt là w_i và h_i, $i = 1, 2, ..., n$, các sản phẩm có thể bị quay 90 độ. Dựa theo cách mô hình hóa bài toán của Huang et al. [**Huang**], ý tưởng để giải bài toán này là ta sẽ nối m mẫu vật liệu lại với nhau theo chiều dọc (Figure 2). Khi này mỗi sản phẩm thứ i sẽ được đặc trưng bởi 5-Tuple $(w_i, h_i, s_i, x_i, y_i)$, lần lượt là chiều rộng, chiều cao, trạng thái quay, hoành độ và tung độ của góc trái dưới sản phẩm này so với góc trái dưới của mẫu vật liệu nằm dưới cùng. s_i là một biến nhị phân, nó sẽ bằng 1 nếu sản phẩm thứ i không bị quay, ngược lại nếu bị quay 90 độ thì $s_i = 0$. Đồng thời, theo Li và Chang [**Li**], với mỗi cặp sản phẩm thứ i và j, $1 \leq i < j \leq n$, ta giới thiệu thêm 2 biến nhị phân lần lượt là u_{ij} và v_{ij}, 2 biến nhị phân này sẽ cho biết vị trí tương đối của 2 sản phẩm tương ứng (Table 2). Ngoài ra, với mỗi cặp sản phẩm i và mẫu vật liệu j, với $i = 1, 2, ..., n$, và $j = 1, 2, ..., m$, ta giới thiệu thêm 1 biến nhị phân Q_{ij}, nó có giá trị 1 khi sản phẩm i nằm bên trong mẫu vật liệu j ($j = 1$ ứng với mẫu vật liệu ở dưới cùng, tăng dần Index khi đi lên), ngược lại bằng 0. Ta giới thiệu thêm biến số thực Y, là giới hạn trên của tất cả các sản phẩm, nghĩa là tung độ của cạnh trên có tung độ cao nhất trong tất cả các sản phẩm (đường màu đỏ trong Figure 2). Như vậy dễ thấy hàm mục tiêu của bài toán sẽ là:

$$\min Y \tag{1}$$

Nhận thấy rằng, số lượng mẫu vật liệu sử dụng sẽ từ $\lceil \frac{Y}{H} \rceil$ trở xuống, nên nếu tối thiểu Y thì cũng chính là tối thiểu số lượng mẫu vật liệu sử dụng. 4 ràng buộc sau sẽ đảm bảo không có 2 sản phẩm nào bị đè lên nhau (Overlap):

$$x_j + w_j s_j + h_j(1 - s_j) \leq x_i + W(u_{ij} + v_{ij}) \qquad i, j = 1, 2, ..., n, \quad i < j \tag{2}$$
$$x_i + w_i s_i + h_i(1 - s_i) \leq x_j + W(1 - u_{ij} + v_{ij}) \qquad i, j = 1, 2, ..., n, \quad i < j \tag{3}$$
$$y_j + h_j s_j + w_j(1 - s_j) \leq y_i + mH(1 + u_{ij} - v_{ij}) \qquad i, j = 1, 2, ..., n, \quad i < j \tag{4}$$
$$y_i + h_i s_i + w_i(1 - s_i) \leq y_j + mH(2 - u_{ij} - v_{ij}) \qquad i, j = 1, 2, ..., n, \quad i < j \tag{5}$$

Trong đó ràng buộc (2) nói rằng, nếu sản phẩm i nằm bên phải sản phẩm j, thì hoành độ của cạnh bên trái sản phẩm i phải không được bé hơn hoành độ của cạnh bên phải sản phẩm j, nếu không thì cả 2 sản phẩm sẽ bị đè lên nhau. Tương tự ràng buộc (3) nói rằng, nếu sản phẩm i nằm bên trái sản phẩm j, thì hoành độ của cạnh bên phải sản phẩm i phải không được lớn hơn hoành độ của cạnh bên trái sản phẩm j. Tiếp theo, ràng buộc (4) nói rằng, nếu sản phẩm i nằm bên trên sản phẩm j, thì tung độ của cạnh bên dưới sản phẩm i phải không được bé hơn

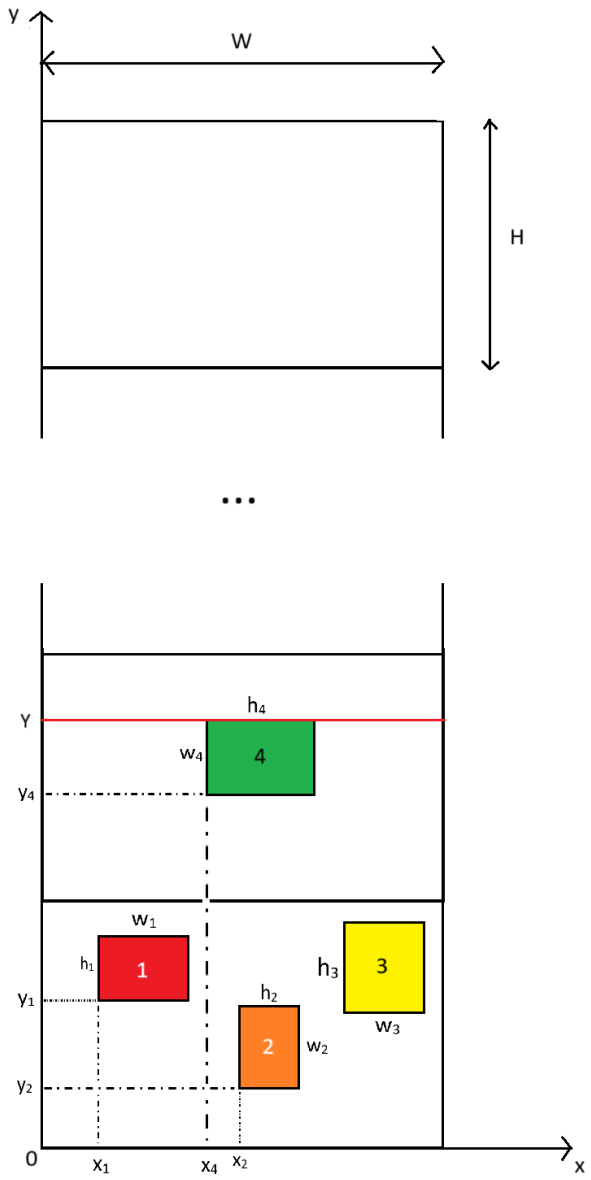


Figure 2: *Minh họa các mẫu vật liệu chồng lên nhau (Item 2 và 4 bị quay 90 độ).*

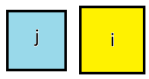
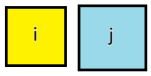
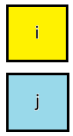
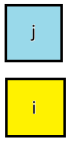
u_{ij}	v_{ij}	Vị trí tương đối của sản phẩm i và sản phẩm j	Minh họa
0	0	sản phẩm i đứng bên phải sản phẩm j	
1	0	sản phẩm i đứng bên trái sản phẩm j	
0	1	sản phẩm i đứng bên trên sản phẩm j	
1	1	sản phẩm i đứng bên dưới sản phẩm j	

Table 2: Mô tả vị trí tương đối của 2 sản phẩm.

tung độ của cạnh bên trên của sản phẩm j. Và cuối cùng, ràng buộc (5) nói rằng, nếu sản phẩm i nằm bên dưới sản phẩm j, thì tung độ của cạnh bên trên sản phẩm i phải không được lớn hơn tung độ của cạnh bên dưới của sản phẩm j. Như vậy, mỗi cặp sản phẩm bất kì sẽ chỉ rơi vào 1 trong 4 trường hợp này.

Ràng buộc sau sẽ ép tất cả các sản phẩm phải nằm ở vị trí thích hợp sao cho không bị vượt khỏi biên bên phải của mẫu vật liệu:

$$x_i + w_i s_i + h_i(1 - s_i) \leq W, \quad i = 1, 2, \ldots, n \tag{6}$$

3 ràng buộc sau sẽ ép mỗi sản phẩm phải nằm gọn trong 1 mẫu vật liệu nào đó, không được nằm giữa biên giới giữa 2 mẫu vật liệu:

$$y_i + h_i s_i + w_i(1 - s_i) \leq \sum_{j=1}^{m} jHQ_{ij}, \qquad i = 1, 2, \ldots, n \tag{7}$$

$$y_i \geq \sum_{j=1}^{m} (j-1)HQ_{ij}, \qquad i = 1, 2, \ldots, n \tag{8}$$

$$\sum_{j=1}^{m} Q_{ij} = 1, \qquad i = 1, 2, \ldots, n \tag{9}$$

Cụ thể, ràng buộc (9) nói rằng, 1 sản phẩm chỉ có thể ở trong đúng 1 mẫu vật liệu. Ràng buộc (7) nói rằng, giả sử sản phẩm này nằm trong mẫu vật liệu thứ j, thì cạnh trên của sản phẩm này phải nằm dưới cạnh trên của mẫu vật liệu đó, nếu không sẽ vượt biên. Tương tự ràng buộc (8) nói rằng, giả sử sản phẩm này nằm trong mẫu vật liệu thứ j, thì cạnh dưới của sản phẩm này phải nằm trên cạnh dưới của mẫu vật liệu j.

Ngoài ra ta cũng phải thêm ràng buộc sau để đảm bảo Y luôn là cận trên của tất cả sản phẩm, tức đảm bảo Y sẽ có giá trị là tung độ của cạnh trên cao nhất trong các sản phẩm:

$$y_i + h_i s_i + w_i(1 - s_i) \leq Y, \quad i = 1, 2, \ldots, n \tag{10}$$

Như vậy mô hình bài toán của chúng ta sẽ bao gồm hàm mục tiêu (1), các ràng buộc (2-10), và cuối cùng là các ràng buộc về kiểu biến:

$$x_i \geq 0, y_i \geq 0, s_i \in \{0, 1\}, \qquad i = 1, 2, \ldots, n \tag{11}$$
$$u_{ij}, v_{ij} \in \{0, 1\}, \qquad i, j = 1, 2, ..., n, \quad i < j \tag{12}$$
$$Q_{ij} \in \{0, 1\}, \qquad i = 1, 2, \ldots, n, \quad j = 1, 2, \ldots, m \tag{13}$$
$$Y \geq 0 \tag{14}$$

Như vậy mô hình của chúng ta có tổng cộng $2n + 1$ biến số thực, $n^2 + nm$ biến nhị phân, và $2n^2 + 3n$ ràng buộc (không tính ràng buộc về kiểu biến). Ở đây ta đã dùng một lượng lớn biến nhị phân, và điều này có thể gây trở ngại trong quá trình tối ưu hóa. Để ý thấy ràng buộc (9) có dạng SOS1 (Special Ordered Set Of Type 1), nghĩa là trong tập hợp m biến gồm Q_{i1} đến Q_{im}, thì chỉ có đúng 1 biến được mang giá trị 1, trong khi tất cả biến còn lại đều bằng 0. Điều này cho thấy, tập m biến gồm Q_{i1} đến Q_{im} chỉ có đúng m trạng thái, đó là $(1, 0, 0, ..., 0), (0, 1, 0, ..., 0), ..., (0, 0, 0, ..., 1)$. Ý tưởng cải thiện ở đây là thay vì dùng One Hot Vector như này, ta sẽ dùng mã hóa nhị phân, như vậy ta chỉ cần dùng $\lceil log_2 m \rceil$ biến nhị phân thay vì m biến nhị phân như trước (Figure 3). Gọi các biến nhị phân mới này là Q'_{ir}, với $i = 1, 2, ..., n$, và $r = 1, 2, ..., \lceil \log_2 m \rceil$. Tuy nhiên bù lại việc cắt giảm được số lượng biến nhị phân, ta cũng phải bù lại khoảng nm biến số thực mới, kí hiệu là λ_{ij}, với $i = 1, 2, ..., n$ và $j = 1, 2, ..., m$. Về bản chất, λ_{ij} hoạt động không khác gì Q_{ij}, nghĩa là mặc dù là biến số thực, nhưng nhờ các ràng buộc mà ta sắp xây dựng sau đây mà nó sẽ luôn chỉ có thể nhận 1 trong 2 giá trị là 0 hoặc 1. Định nghĩa tập A_r là tập gồm các số tự nhiên từ 1 đến m, sao cho dạng nhị phân của chúng có chữ số thứ r từ phải sang có giá trị bằng 1, ví dụ $A_3 = \{4, 5, 6, 7, 12, 13, 14, ...\}$ (Figure 4). 3 ràng buộc cũ sẽ bị thay thế là (7), (8), (9), thay thế cho chúng là 4 ràng buộc mới sau:

$$y_i + h_i s_i + w_i(1 - s_i) \leq \sum_{j=1}^{m} jH\lambda_{ij} \qquad i = 1, 2, \ldots, n \tag{15}$$
$$y_i \geq \sum_{j=1}^{m} (j - 1)H\lambda_{ij} \qquad i = 1, 2, \ldots, n \tag{16}$$
$$\sum_{j=1}^{m} \lambda_{ij} = 1 \qquad i = 1, 2, \ldots, n \tag{17}$$
$$\sum_{j \in A_r} \lambda_{ij} = Q'_{ir} \qquad i = 1, 2, \ldots, n, \quad r = 1, 2, \ldots, \lceil \log_2 m \rceil \tag{18}$$

Đồng thời, ta cũng sẽ không quên thay thế ràng buộc kiểu biến (13) thành 2 ràng buộc mới sau:

$$\lambda_{ij} \geq 0, \qquad i = 1, 2, \ldots, n, \quad j = 1, 2, \ldots, m \tag{19}$$
$$Q'_{ir} \in \{0, 1\}, \qquad i = 1, 2, \ldots, n, \quad r = 1, 2, \ldots, \lceil \log_2 m \rceil \tag{20}$$

8 biến nhị phân → 3 biến nhị phân

Trạng thái	Q_{i1}	Q_{i2}	Q_{i3}	Q_{i4}	Q_{i5}	Q_{i6}	Q_{i7}	Q_{i8}	Q'_{i3}	Q'_{i2}	Q'_{i1}
0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	1
2	0	0	1	0	0	0	0	0	0	1	0
3	0	0	0	1	0	0	0	0	0	1	1
4	0	0	0	0	1	0	0	0	1	0	0
5	0	0	0	0	0	1	0	0	1	0	1
6	0	0	0	0	0	0	1	0	1	1	0
7	0	0	0	0	0	0	0	1	1	1	1

Figure 3: *Minh họa cách giảm số lượng biến nhị phân khi xuất hiện ràng buộc SOS1.*

A_3	Chữ số thứ 3 từ phải sang ↓
4	100
5	101
6	110
7	111
12	1100
13	1101
14	1110
15	1111
20	10100
21	10101
...	

Figure 4: *Minh họa cách xác định tập A_3.*

Như vậy mô hình cuối cùng sẽ chỉ còn $n^2 + n\lceil log_2 m\rceil$ biến nhị phân, tuy nhiên số biến thực tăng lên thành $\frac{1}{2}n^2 + \frac{3}{2}n + 1$, và số ràng buộc tăng thành $2n^2 + 3n + n\lceil log_2 m\rceil$. Ta có thể giải mô hình này bằng các MILP (Mixed Integer Linear Programming) Solver.

3.2 Mô hình hóa bài toán trong trường hợp mẫu vật liệu có nhiều kích thước khác nhau với số lần sử dụng không giới hạn

Nhắc lại trong trường hợp này, thì các Pattern sẽ có chi phí bằng diện tích của mẫu vật liệu dùng để cắt nó. Giả sử ta có m loại mẫu vật liệu, mỗi loại có chiều rộng và chiều cao lần lượt là W_i và H_i, với $i = 1, 2, ..., m$. Ta được yêu cầu cắt ra n loại sản phẩm với kiểu cắt 2-Stage Guillotine có xén, mỗi loại có chiều rộng, chiều cao và số lượng yêu cầu là w_i, h_i, và d_i, với $i = 1, 2, ..., n$. Các sản phẩm phải được đảm bảo là không bị quay 90 độ, và các loại sản phẩm đã được sắp xếp sao cho chiều rộng không tăng, tức $w_1 \geq w_2 \geq ... \geq w_n$. Định nghĩa n' là tổng số sản phẩm cần cắt, tức $n' = \sum_{i=1}^{n} d_i$. Dựa theo cách mô hình hóa bài toán của Furini và Malaguti [**Furini**], ý tưởng để giải bài toán 2D CSP trong trường hợp này là sử dụng phương pháp gần giống như phương pháp phân dải (Method Of Stripe) trong phần mô hình hóa bài toán phụ của giải thuật Column Generation (4.1.2). Về cơ bản, mỗi sản phẩm thứ i trong n' sản phẩm sẽ ứng với 1 dải (Stripe), kí hiệu là s_i, có chiều rộng bằng chiều rộng của sản phẩm thứ i, và chiều cao bằng chiều cao của mẫu vật liệu chứa nó. Quy ước các sản phẩm được đặt vào trong dải theo thứ tự từ dưới lên trên, với chiều rộng của các sản phẩm giảm dần, sản phẩm nằm dưới cùng sẽ có chiều rộng lớn nhất và bằng chiều rộng của dải. Định nghĩa a_i bằng tổng số lượng sản phẩm cần cắt có chiều rộng không bé hơn chiều rộng của sản phẩm loại i, nghĩa là $a_i = \sum_{j=1}^{i} d_j$, lưu ý ta cũng định nghĩa $a_0 = 0$. Đồng thời với mỗi sản phẩm thứ i ($i = 1, 2, ..., n'$), ta định nghĩa giá trị b_i ứng với nó bằng chính loại sản phẩm của nó (xem Figure 5). Giả sử với mỗi loại mẫu vật liệu i ($i = 1, 2, ..., m$), ta "phân thân" nó thành n' bản, khi này chắc chắn một điều là ta sẽ không thể thiếu mẫu vật liệu trong quá trình cắt các sản phẩm, đó là bởi vì chúng ta chỉ cần cắt n' sản phẩm trong khi bây giờ có tới mn' mẫu vật liệu, nên thậm chí mỗi 1 mẫu vật liệu chỉ cắt đúng 1 sản phẩm thì cũng không sợ hết. Với mỗi mẫu vật liệu thứ j thuộc loại i, $i = 1, 2, ..., m$, $j = 1, 2, ..., n'$, ta sẽ giới thiệu 1 biến nhị phân q_{ij}, nó sẽ bằng 1 nếu mẫu vật liệu này thực sự được sử dụng, tức có ít nhất 1 sản phẩm được cắt ra từ nó, ngược lại bằng 0. Đồng thời ta cũng định nghĩa biến nhị phân y_{ij}, $i = 1, 2, ..., m$, $j = 1, 2, ..., n'$, nó có giá trị 1 nếu dải s_j được nằm trong 1 mẫu vật liệu loại i, ngược lại bằng 0. Ngoài ra, ta sẽ định nghĩa thêm 1 biến số nguyên mới x_{ijk}, $i = 1, 2, ..., m$, $j = 1, 2, ..., n' - 1$, $k = b_j, ..., n$, nó tương đương số sản phẩm loại k nằm trong dải s_j, trừ sản phẩm nằm dưới cùng, và dải s_j này phải nằm trong 1 mẫu vật liệu loại i nào đó. Cuối cùng, ta định nghĩa biến nhị phân z_{ijk}, với $i = 1, 2, ..., m$, $j = 1, 2, ..., n' - 1$, $k = j + 1, ..., n'$, nó sẽ bằng 1 nếu dải s_k được đặt vào mẫu vật liệu thứ j thuộc loại i, và không phải là dải nằm bên trái cùng, ngược lại bằng 0. Đến đây ta cũng đã có đủ tiền đề để thiết lập hàm mục tiêu cho bài toán như sau:

$$\min \sum_{i=1}^{m} \left(W_i H_i \sum_{j=1}^{n'} q_{ij} \right) \tag{21}$$

Dễ thấy biểu thức trên chính là tổng diện tích của tất cả các mẫu vật liệu thật sự được sử dụng. Tiếp theo ta cần một ràng buộc để tổng số sản phẩm cắt ra phải đủ số lượng yêu cầu:

$$\sum_{i=1}^{m} \left(\left(\sum_{j=1}^{a_k - 1} x_{ijk} \right) + \left(\sum_{j=a_{k-1}+1}^{a_k} y_{ij} \right) \right) \geq d_k, \quad k = 1, 2, \ldots, n \tag{22}$$

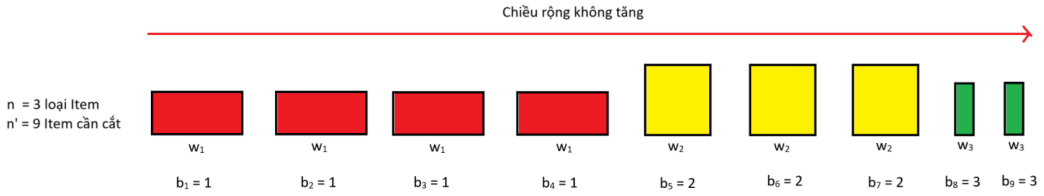


Figure 5: *Minh họa cách xác định b_i.*

Bản chất của biểu thức ở vế trái trong ràng buộc trên là tổng số lượng sản phẩm loại k không nằm dưới cùng cộng với tổng số lượng sản phẩm loại k nằm dưới cùng, do đó khi cộng lại chúng sẽ ra tổng số lượng sản phẩm loại k được cắt. Ràng buộc tiếp theo sẽ ép các sản phẩm phải được xếp trong các dải sao cho tổng chiều cao của các sản phẩm trong cùng 1 dải là không vượt quá chiều cao của dải đó:

$$\sum_{k=b_j}^{n} h_k x_{ijk} \leq (L_i - h_{b_j}) y_{ij}, \quad i = 1, 2, \ldots, m, \quad j = 1, 2, \ldots, n' - 1 \tag{23}$$

Ràng buộc tiếp theo sẽ ép một dải bất kì phải rơi vào đúng 1 trong 3 trường hợp, hoặc là không được chọn vào mẫu vật liệu nào, hoặc là được chọn và trở thành dải nằm ở bên trái cùng của 1 mẫu vật liệu, hoặc là được chọn nhưng không phải dải nằm ở bên trái cùng của mẫu vật liệu chứa nó:

$$\left(\sum_{k=1}^{j-1} z_{ikj} \right) + q_{ij} = y_{ij}, \quad i = 1, 2, \ldots, m, \quad j = 1, 2, \ldots, n' \tag{24}$$

Ràng buộc sau sẽ ép tổng chiều rộng của các dải trong cùng 1 mẫu vật liệu không được vượt quá chiều rộng của mẫu vật liệu đó:

$$\sum_{k=j+1}^{n'} w_{b_k} z_{ijk} \leq (W_i - w_{b_j}) q_{ij}, \quad i = 1, 2, \ldots, m, \quad j = 1, 2, \ldots, n' - 1 \tag{25}$$

Cuối cùng ta có các ràng buộc về kiểu biến như sau:

$$y_{ij}, q_{ij} \in \{0, 1\}, \qquad i = 1, 2, \ldots, m, \quad j = 1, 2, \ldots, n' \tag{26}$$

$$x_{ijk} \in \mathbb{N}, \qquad i = 1, 2, ..., m, \quad j = 1, 2, ..., n' - 1, \quad k = b_j, ..., n \tag{27}$$

$$z_{ijk} \in \{0, 1\}, \qquad i = 1, 2, ..., m, \quad k = 1, 2, ..., n', \quad j = 1, 2, ..., k - 1 \tag{28}$$

Dễ thấy đây là 1 mô hình ILP với số lượng biến là $O(mn'^2)$, và số lượng ràng buộc là $O(mn')$. Với số lượng biến và ràng buộc ở mức này, bài toán có thể trở nên khá phức tạp và tốn nhiều thời gian tính toán khi kích thước tăng lên.

3.3 Mô hình hóa bài toán trong trường hợp chỉ có hữu hạn các mẫu vật liệu với kích thước khác nhau

Giả sử ta có m mẫu vật liệu, chiều rộng và chiều cao của chúng lần lượt là W_i và H_i, $i = 1, 2, ..., m$. Yêu cầu cắt ra n sản phẩm, mỗi sản phẩm có chiều rộng và chiều cao lần lượt là w_i và h_i,

$i = 1, 2, ..., n$. Kiểu cắt của ta là sản phẩm cắt ở vị trí nào cũng được, miễn là không bị quay góc 90 độ, và không đè lên nhau. Ta sẽ giới thiệu thêm biến nhị phân z_{ij}, $i = 1, 2, ..., n$, $j = 1, 2, ..., m$, nó sẽ có giá trị bằng 1 nếu sản phẩm thứ i nằm trong mẫu vật liệu thứ j, ngược lại bằng 0. Đồng thời để mô tả vị trí của các sản phẩm, ta cũng sẽ giới thiệu thêm 2 biến số thực là x_i và y_i, $i = 1, 2, ..., n$, lần lượt là hoành độ và tung độ của góc trái dưới của sản phẩm, so với góc trái dưới của mẫu vật liệu chứa nó. Với mỗi cặp sản phẩm thứ i và j, $1 \leq i < j \leq n$, ta giới thiệu 1 biến nhị phân b_{ij}, nó sẽ bằng 1 nếu sản phẩm i nằm trong mẫu vật liệu có Index nhỏ hơn Index của mẫu vật liệu chứa sản phẩm j, và sẽ bằng 0 nếu sản phẩm i nằm trong mẫu vật liệu có Index lớn hơn Index của mẫu vật liệu chứa sản phẩm j, trong trường hợp cả hai đều nằm trong cùng 1 mẫu vật liệu, thì giá trị b_{ij} là không quan trọng, ngoài ra ta cũng dùng tới cả cặp biến nhị phân u_{ij} và v_{ij} để mô tả vị trí tương đối của 2 sản phẩm (Table 2). Đồng thời với mỗi mẫu vật liệu thứ i ($i = 1, 2, ..., m$), ta sẽ giới thiệu thêm 1 biến nhị phân q_i, nó sẽ bằng 1 nếu mẫu vật liệu này được sử dụng, tức có ít nhất 1 sản phẩm được cắt ra từ nó, ngược lại bằng 0. Bởi vì ta muốn tối thiểu hóa tổng diện tích của các mẫu vật liệu sử dụng, nên hàm mục tiêu sẽ như sau:

$$\min \sum_{i=1}^{m} W_i H_i q_i \tag{29}$$

8 ràng buộc sau là để cho 2 sản phẩm bất kì nằm trong cùng 1 mẫu vật liệu sẽ không được phép đè lên nhau:

$$x_j + w_j \leq x_i + M_1(u_{ij} + v_{ij}) + M_2 \sum_{k=1}^{m} k(z_{ik} - z_{jk}) + M_3 b_{ij}, \qquad i, j = 1, 2, \ldots, n, \quad i < j \tag{30}$$

$$x_j + w_j \leq x_i + M_1(u_{ij} + v_{ij}) - M_2 \sum_{k=1}^{m} k(z_{ik} - z_{jk}) + M_3(1 - b_{ij}), \qquad i, j = 1, 2, \ldots, n, \quad i < j \tag{31}$$

$$x_i + w_i \leq x_j + M_1(1 - u_{ij} + v_{ij}) + M_2 \sum_{k=1}^{m} k(z_{ik} - z_{jk}) + M_3 b_{ij}, \qquad i, j = 1, 2, \ldots, n, \quad i < j \tag{32}$$

$$x_i + w_i \leq x_j + M_1(1 - u_{ij} + v_{ij}) - M_2 \sum_{k=1}^{m} k(z_{ik} - z_{jk}) + M_3(1 - b_{ij}), \quad i, j = 1, 2, \ldots, n, \quad i < j \tag{33}$$

$$y_j + h_j \leq y_i + M_1(1 + u_{ij} - v_{ij}) + M_2 \sum_{k=1}^{m} k(z_{ik} - z_{jk}) + M_3 b_{ij}, \qquad i, j = 1, 2, \ldots, n, \quad i < j \tag{34}$$

$$y_j + h_j \leq y_i + M_1(1 + u_{ij} - v_{ij}) - M_2 \sum_{k=1}^{m} k(z_{ik} - z_{jk}) + M_3(1 - b_{ij}), \quad i, j = 1, 2, \ldots, n, \quad i < j \tag{35}$$

$$y_i + h_i \leq y_j + M_1(2 - u_{ij} - v_{ij}) + M_2 \sum_{k=1}^{m} k(z_{ik} - z_{jk}) + M_3 b_{ij}, \qquad i, j = 1, 2, \ldots, n, \quad i < j \tag{36}$$

$$y_i + h_i \leq y_j + M_1(2 - u_{ij} - v_{ij}) - M_2 \sum_{k=1}^{m} k(z_{ik} - z_{jk}) + M_3(1 - b_{ij}), \quad i, j = 1, 2, \ldots, n, \quad i < j \tag{37}$$

Ở đây, M_1, M_2 và M_3 đều là các hằng số rất lớn (Big-M), trong đó $M_1 << M_2 << M_3$. 2 ràng buộc đầu tiên nói rằng, nếu sản phẩm i và j nằm trong cùng 1 mẫu vật liệu và sản phẩm i nằm bên phải sản phẩm j, thì hoành độ của cạnh bên trái sản phẩm i phải không được bé hơn hoành độ của cạnh bên phải sản phẩm j, nếu không thì cả 2 sản phẩm sẽ bị đè lên nhau. Tương tự 2 ràng buộc tiếp theo nói rằng, nếu sản phẩm i nằm bên trái sản phẩm j trong cùng mẫu vật liệu, thì hoành độ của cạnh bên phải sản phẩm i phải không được lớn hơn hoành độ của cạnh bên trái sản phẩm j. Tiếp theo, ràng buộc thứ 5 và 6 nói rằng, nếu sản phẩm i nằm bên trên sản phẩm j trong cùng mẫu vật liệu, thì tung độ của cạnh bên dưới sản phẩm i phải không được bé hơn tung độ của cạnh bên trên của sản phẩm j. Và cuối cùng, ràng buộc thứ 7 và 8 nói rằng, nếu sản phẩm i nằm bên dưới sản phẩm j trong cùng mẫu vật liệu, thì tung độ của cạnh bên trên sản phẩm i phải không được lớn hơn tung độ của cạnh bên dưới của sản phẩm j.

2 ràng buộc sau đây sẽ ép các sản phẩm phải nằm gọn bên trong mẫu vật liệu chứa chúng, không được tràn biên bên phải và bên trên của mẫu vật liệu:

$$x_i + w_i \leq \sum_{j=1}^{m} z_{ij} W_j, \qquad i = 1, 2, \ldots, n \tag{38}$$

$$y_i + h_i \leq \sum_{j=1}^{m} z_{ij} H_j, \qquad i = 1, 2, \ldots, n \tag{39}$$

Ràng buộc tiếp theo sẽ buộc mỗi sản phẩm chỉ được phép nằm trong 1 mẫu vật liệu duy nhất:

$$\sum_{j=1}^{m} z_{ij} = 1, \qquad i = 1, 2, \ldots, n \tag{40}$$

2 ràng buộc sau sẽ ép q_j phải bằng 1 nếu có ít nhất 1 biến z_{ij} có giá trị bằng 1, với $1 \leq i \leq n$, nghĩa là q_j sẽ bằng 1 khi và chỉ khi mẫu vật liệu thứ j chứa ít nhất 1 sản phẩm, ngược lại $q_j = 0$:

$$q_j \leq \sum_{i=1}^{n} z_{ij}, \qquad j = 1, 2, \ldots, m \tag{41}$$

$$\sum_{i=1}^{n} z_{ij} \leq n \cdot q_j \qquad j = 1, 2, \ldots, m \tag{42}$$

Như vậy, mô hình bài toán trong trường hợp này sẽ bao gồm hàm mục tiêu (29), các ràng buộc (30-42), kèm theo các ràng buộc về kiểu biến như sau:

$$x_i \geq 0, \quad y_i \geq 0, \qquad i = 1, 2, \ldots, n \tag{43}$$
$$u_{ij}, v_{ij}, b_{ij} \in \{0, 1\}, \qquad i, j = 1, 2, \ldots, n, \quad i < j \tag{44}$$
$$z_{ij} \in \{0, 1\}, \qquad i = 1, 2, \ldots, n, \quad j = 1, 2, \ldots, m \tag{45}$$
$$q_i \in \{0, 1\}, \qquad i = 1, 2, \ldots, m \tag{46}$$

Như vậy, mô hình của chúng ta tổng cộng có $2n$ biến số thực và $\frac{3}{2}n^2 - \frac{3}{2}n + mn + m$ biến nhị phân, đồng thời có tới $4n^2 - n + 2m$ ràng buộc. Với số lượng biến nhị phân và ràng buộc lớn như thế này, thì cho dù ta có thực hiện các biện pháp giảm số lượng biến nhị phân thì bài toán vẫn rất khó để giải bằng những phương pháp thông thường. Do đó, đối với trường hợp này của bài toán 2D CSP, ta ưu tiên sử dụng các giải thuật Heuristic.

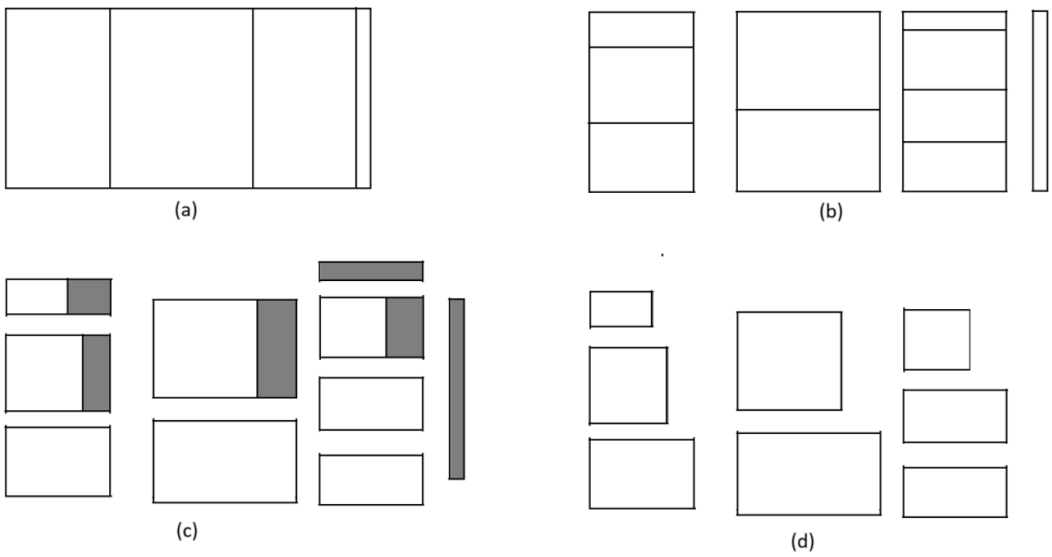


Figure 6: *(a) Mẫu vật liệu ban đầu. (b) Sau giai đoạn cắt đầu tiên. (c) Sau giai đoạn cắt thứ hai. (d) Sau khi cắt xén và loại bỏ vật liệu thừa.*

4 Các giải thuật

Ở section này, nhóm sẽ giới thiệu 4 thuật toán nổi bật để giải bài toán 2D CSP, gồm giải thuật sinh cột (Column Generation), giải thuật tham lam (Greedy Algorithm), giải thuật tôi luyện mô phỏng (Simulated Annealing) và giải thuật học tăng cường (Reinforcement Learning). Trong đó giải thuật đầu là giải thuật chính xác trong trường hợp kiểu cắt là 2-Stage Guillotine có xén, còn các giải thuật còn lại thuộc loại giải thuật gần đúng (Heuristic) và có thể áp dụng cho nhiều dạng bài toán hơn. Kết quả so sánh giữa các giải thuật sẽ được cho ở section 5, trong đó nhóm chạy các giải thuật Heuristic trên môi trường gymnasium, còn giải thuật sinh cột và phương pháp Simplex dựa theo các mô hình trong section 3 sẽ được chạy trên phần mềm độc lập do nhóm tạo ra.

4.1 Giải thuật Column Generation

4.1.1 Ý tưởng

Giả sử trong bài toán 2D CSP của ta, số mẫu vật liệu là vô hạn và có kích thước cố định, kiểu cắt là thuộc loại 2-Stage Guillotine có xén, tức sau 2 giai đoạn đầu tiên, ta còn làm thêm 1 bước cuối là cắt xén bớt các mảnh hình chữ nhật đã cắt ra để có được kích thước các sản phẩm theo yêu cầu (Figure 6). Đồng thời giả định rằng các sản phẩm không bị quay 90 độ so với mẫu vật liệu và trong giai đoạn đầu tiên ta sẽ cắt theo phương dọc.

Việc triển khai thuật toán Column Generation cho bài toán 2D CSP ở trường hợp trên cũng giống như trong 1D CSP, chỉ khác ở chỗ là việc giải quyết bài toán phụ (Subproblem) là tương đối phức tạp, mô hình của bài toán phụ sẽ được cho trong phần tiếp theo (4.1.2).

Giả sử mẫu vật liệu có chiều rộng và chiều cao lần lượt là W và H. Ta muốn cắt ra n loại sản phẩm, mỗi loại sẽ có chiều rộng, chiều cao và số lượng yêu cầu tối thiểu lần lượt là w_i, h_i và d_i,

với $i = 1, 2, ..., n$. Đầu tiên, xét tới cách tiếp cận thông thường là ta sẽ tạo ra tất cả các Pattern có thể có, ta quy ước 2 Pattern bất kì sẽ chỉ khác nhau khi số lượng các sản phẩm mỗi loại trong chúng là khác nhau, vị trí cắt không ảnh hưởng. Giả sử có tổng cộng p Pattern khác nhau, với mỗi Pattern thứ i ($i = 1, 2, ..., p$), sẽ được đặc trưng bởi 1 Vector a_i gồm n phần tử, phần tử thứ j ($j = 1, 2, ..., n$) kí hiệu là a_{ij}, chính bằng số lượng của sản phẩm loại j trong Pattern thứ i. Gọi số lượng Pattern thứ i dùng để cắt là x_i, ta dễ dàng lập được hàm mục tiêu và các ràng buộc của bài toán như sau:

$$\min \sum_{i=1}^{p} x_i \tag{47}$$

$$\text{subject to} \quad \sum_{i=1}^{p} a_{ij}\, x_i \geq d_j, \quad j = 1, 2, \ldots, n, \tag{48}$$

$$x_i \in \mathbb{N}, \quad i = 1, 2, \ldots, p. \tag{49}$$

Vấn đề ở đây là nếu ta tạo ra tất cả Pattern trong bước đầu tiên, thì có thể trong quá trình tìm ra kết quả tối ưu, sẽ có những Pattern mà ta không đụng tới, như vậy sẽ gây ra một lượng lãng phí đáng kể về thời gian tính toán và bộ nhớ. Thay vào đó, với giải thuật Column Generation, ta sẽ bắt đầu với chỉ n Pattern, mỗi Pattern tạo thành 1 cột và n Pattern này sẽ tạo thành 1 ma trận vuông kích thước $n \times n$, tạm gọi là A. Thông thường để khởi đầu dễ dàng, ta sẽ chọn A là ma trận đường chéo:

$$A = \begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{pmatrix} \tag{50}$$

Nếu muốn đơn giản, ta có thể chọn A là ma trận đồng nhất (Identity), nhưng thay vì làm như vậy, ta sẽ chọn A là ma trận đường chéo với các phần tử trên đường chéo chính là lớn nhất có thể, vì nó sẽ gần với nghiệm tối ưu hơn. Dễ thấy các phần tử trên đường chéo chính của A khi này có thể được tính bằng công thức sau:

$$a_{ii} = \left\lfloor \frac{W}{w_i} \right\rfloor \times \left\lfloor \frac{H}{h_i} \right\rfloor, \quad i = 1, 2, \ldots, n \tag{51}$$

Theo giải thuật Column Generation, trước tiên ta cần xây dựng mô hình cho bài toán RMP - Restricted Master Problem, nói cách khác là bài toán chính đã bị giới hạn chỉ còn một vài cột. Giả sử thời điểm hiện tại A có q cột, với $n \leq q \leq p$, tức hiện tại ta đã tạo ra được q Pattern, số lượng mỗi Pattern sử dụng sẽ được cho bởi Vector x, có các phần tử là x_i, với $i = 1, 2, ..., q$. Gọi d là Vector gồm các phần tử là số lượng yêu cầu tối thiểu của mỗi sản phẩm, tức các phần tử của d sẽ là d_i, với $i = 1, 2, ..., n$, Như vậy ta có thể biểu diễn RMP dưới dạng ma trận như sau:

$$\min \quad 1^T x \tag{52}$$

$$\text{subject to} \quad Ax \geq d, \tag{53}$$

$$x \in \mathbb{N}^q \tag{54}$$

Giả sử sau khi giải bài toán RMP đã được Relax (bỏ đi ràng buộc $x \in \mathbb{N}^q$ và thay bằng $x \geq 0$), ta được ma trận cơ sở (Basis) tối ưu là ma trận vuông B kích thước $n \times n$ nằm trong

A. Tưởng tượng ma trận chứa tất cả các Pattern có thể có là A', dễ thấy A' chứa B, khi này Reduced Cost sẽ là $r = 1^T - 1^T B^{-1} A'$. Theo quy tắc chọn biến xâm nhập (Entering Variable) của Dantzig, ta sẽ chọn biến xâm nhập ứng với Reduced Cost âm nhất, nếu không có Reduced Cost nào âm thì dừng thuật toán và kết luận kết quả hiện tại là tối ưu. Câu hỏi đặt ra ở đây là liệu có cách nào khác để không cần phải xác định toàn bộ ma trận A', nói cách khác là không cần lặp qua tất cả các Pattern, mà vẫn xác định được Pattern nào có Reduced Cost nhỏ nhất? Quả thật, nếu định nghĩa A'_i là Vector cột thứ i trong A', thì bản chất là ta đang tối ưu hàm mục tiêu sau:

$$\min_{i=1,2,\dots,p} \left(1^T - 1^T B^{-1} A'_i\right) \tag{55}$$

Nếu đặt $\lambda^T = 1^T B^{-1}$, thì hàm mục tiêu trên có thể viết lại thành:

$$\max_{i=1,2,\dots,p} \lambda^T A'_i \tag{56}$$

Ở đây λ được gọi là biến đối ngẫu (Dual Variable), giá trị của nó có thể được xác định thông qua việc giải bài toán đối ngẫu của Relaxed RMP. Bài toán phụ sẽ là tìm Pattern để hàm mục tiêu trên đạt giá trị lớn nhất. Sau khi đã giải quyết được bài toán phụ và tìm ra Pattern ứng với Reduced Cost âm nhất, thì ta sẽ tiến hành kết nạp Pattern đó thành 1 cột của RMP, đồng thời giới thiệu thêm 1 biến mới x_{q+1} ứng với số lượng Pattern này được sử dụng, và tiến hành tiếp tục giải lại bài toán RMP. Ngược lại, nếu Pattern tìm được có Reduced Cost nhỏ nhất nhưng lại từ 0 trở lên, tức giá trị tối ưu của hàm mục tiêu (56) là từ 1 trở xuống, thì ta sẽ dừng thuật toán vì khi này không thể tối ưu thêm được nữa. Bởi vì, những gì ta làm đến giờ là giải phiên bản đã được Relax của RMP, kết quả của phiên bản này sẽ là cận dưới của bài toán ban đầu, nên nếu ta không thể tối ưu thêm bài toán đã được Relax thì cũng đồng nghĩa với việc không thể tối ưu thêm bài toán gốc. Như vậy, sau khi dừng thuật toán, ta sẽ có một tập hợp các Pattern cần thiết, và khi này ta có thể giải bài toán RMP theo cách thông thường bằng giải thuật Simplex kết hợp Branch-and-Bound, hoặc sử dụng các thuật toán khác.

4.1.2 Mô hình hóa bài toán phụ của giải thuật Column Generation

Nhắc lại ở trường hợp đang xét, kiểu cắt là thuộc loại 2-Stage Guillotine có xén, đồng thời giả định rằng các sản phẩm không bị quay 90 độ so với mẫu vật liệu và trong giai đoạn đầu tiên ta sẽ cắt theo phương dọc (Figure 7).

Phương pháp mà ta dùng để mô hình hóa bài toán phụ được gọi là phương pháp phân dải hay Method Of Stripe (Figure 8) theo Novianingsih et al. [**Novianingsih**]. Một dải (Stripe) sẽ có được bằng cách cắt mẫu vật liệu theo chiều dọc thẳng xuống. Mỗi một loại sản phẩm i $(i = 1, 2, ..., n)$, ta sẽ chia mẫu vật liệu thành các dải với chiều rộng là w_i và chiều cao là H, dễ thấy sẽ có tối đa $\lfloor \frac{W}{w_i} \rfloor$ dải. Kí hiệu các dải này là s_{ij}, với $i = 1, 2, ..., n, j = 1, 2, ..., \lfloor \frac{W}{w_i} \rfloor$. Với mỗi dải s_{ij}, ta sẽ giới thiệu thêm 1 biến nhị phân z_{ij}, nó sẽ có giá trị là 1 nếu dải s_{ij} được đưa vào mẫu vật liệu, ngược lại có giá trị 0. Gọi $A_j = \{i \in \mathbb{N} \mid 1 \leq i \leq n, w_i \leq w_j\}$, với $j = 1, 2, ..., n$, tức A_j là tập hợp Index của các loại sản phẩm có chiều rộng không vượt quá chiều rộng của sản phẩm loại j. Giới thiệu thêm các biến số nguyên y_{ijk}, nó sẽ có giá trị bằng số lượng sản phẩm loại i được đặt vào trong dải s_{jk}, với $i \in A_j$. Đến đây ta đã đủ dữ kiện để xây dựng hàm mục tiêu cho bài toán phụ như sau:

$$\min \sum_{j=1}^{n} \sum_{k=1}^{\lfloor \frac{W}{w_j} \rfloor} \sum_{i \in A_j} \lambda_i \cdot y_{ijk} \tag{57}$$

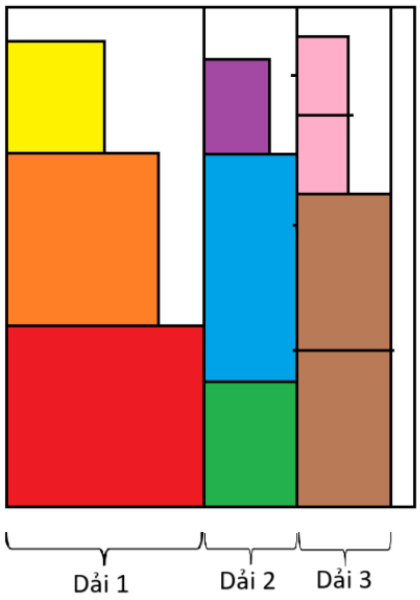


Figure 7: *Minh họa các dải (Stripe) trong kiểu cắt 2-Stage Guillotine có xén.*

Ở đây λ_i như đã biết là giá trị đối ngẫu, nó được nhân với tổng số lượng của sản phẩm loại i trong Pattern mới. Tiếp theo ta cần ràng buộc là tổng chiều rộng các dải được chọn không vượt quá chiều rộng của mẫu vật liệu:

$$\sum_{i=1}^{n} \sum_{j=1}^{\lfloor \frac{W}{w_i} \rfloor} w_i z_{ij} \leq W \tag{58}$$

Ràng buộc tiếp theo là để các sản phẩm đặt trong cùng 1 dải sẽ có tổng chiều cao không vượt quá chiều cao của dải = chiều cao của mẫu vật liệu:

$$\sum_{i \in A_j} h_i y_{ijk} \leq H z_{jk}, \quad j = 1, 2, \ldots, n, \quad k = 1, 2, \ldots, \left\lfloor \frac{W}{w_j} \right\rfloor \tag{59}$$

Như vậy mô hình bài toán phụ của giải thuật Column Generation sẽ bao gồm hàm mục tiêu (57), với các ràng buộc (58), (59), kèm theo các ràng buộc về kiểu biến như sau là đủ:

$$y_{ijk} \in \mathbb{N}, \quad j = 1, 2, \ldots, n, \quad i \in A_j, \quad k = 1, 2, \ldots, \left\lfloor \frac{W}{w_j} \right\rfloor \tag{60}$$

$$z_{ij} \in \{0, 1\}, \quad i = 1, 2, \ldots, n, \quad j = 1, 2, \ldots, \left\lfloor \frac{W}{w_i} \right\rfloor \tag{61}$$

Ta có thể giải dễ dàng mô hình quy hoạch nguyên trên bằng các ILP Solver.

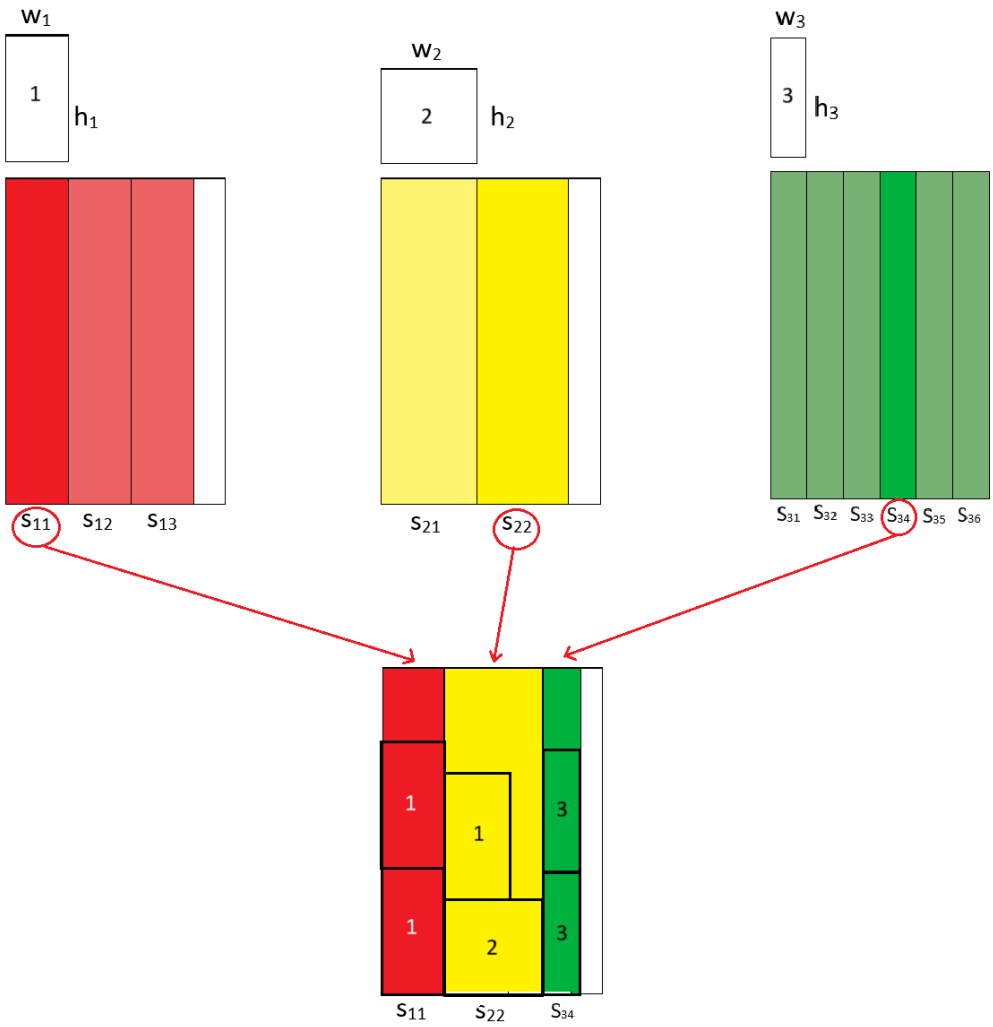


Figure 8: *Minh họa cơ chế chọn dải để đưa vào mẫu vật liệu.*

4.1.3 Mã giả

Dưới đây là phần mã giả cho thuật toán Column Generation:

Input: Kích thước mẫu vật liệu, danh sách kích thước và yêu cầu tối thiểu của mỗi sản phẩm.

Output: Danh sách các Pattern cần thiết kèm theo số lượng của chúng.

1. Định nghĩa hàm mục tiêu và các ràng buộc ban đầu của bài toán Relaxed Master Problem (RMP) và bài toán phụ.

2. Khởi tạo tập A với các Pattern ban đầu.

3. **while** True

- **3.1.** Giải bài toán đối ngẫu của Relaxed RMP ứng với A để tìm giá trị của biến đối ngẫu λ.
- **3.2.** Giải bài toán phụ với giá trị đối ngẫu λ vừa tính được.
- **3.3.** `X` $\leftarrow$ giá trị tối ưu của hàm mục tiêu của bài toán phụ.
- **3.4.** **Nếu** `X > 1` **thì** thêm Pattern tìm được vào A và giới thiệu biến mới tương ứng với số lượng của nó. **Ngược lại**, thoát vòng lặp.

4. Giải lại bài toán RMP với các Pattern hiện có, được Vector x lưu trữ số lượng sử dụng của các Pattern.

5. Trả về tập con của A ứng với các Pattern có số lượng sử dụng > 0, cùng với x.

Algorithm 1: Column Generation

4.1.4 Ưu và nhược điểm của giải thuật Column Generation

Việc lựa chọn cách giải quyết một bài toán là rất quan trọng trong lĩnh vực tối ưu hóa, đặc biệt ảnh hưởng sâu sắc đến hiệu quả của quá trình tìm kiếm nghiệm tối ưu. Giải thuật sinh cột (Column Generation) đã thể hiện được tính ưu việt của nó trong nhiều trường hợp, tuy nhiên, nó cũng còn tồn tại một số hạn chế nhất định. Dưới đây, chúng ta sẽ phân tích những ưu điểm và nhược điểm của giải thuật này.

Ưu điểm

- **Tiết kiệm bộ nhớ và tài nguyên:** Vì khi bắt đầu, ta chỉ mới làm việc với những Pattern cơ bản thay vì xét toàn bộ các Pattern có thể có. Và trong quá trình giải bài toán, chỉ có các Pattern tiềm năng mới được thêm vào RMP, giúp giảm số biến cần xử lý.
- **Hiệu quả với các bài toán lớn:** Vì đôi khi trong một bài toán có hàng ngàn Pattern khác nhau, nhưng chỉ số ít trong chúng là đóng góp đến nghiệm tối ưu, nên việc chỉ thêm các Pattern tối ưu tại mỗi vòng lặp sẽ giúp ta tránh tạo ra các Patter không cần thiết.

- **Dễ dàng mở rộng:** Column Generation có thể được kết hợp với các giải thuật tối ưu khác như Branch-and-Bound hoặc Branch-and-Cut, hay các giải thuật liên quan đến việc xóa bỏ các cột không còn được sử dụng trong RMP, từ đó mở rộng khả năng giải quyết các bài toán phức tạp mà vẫn tiết kiệm được tài nguyên.

Nhược điểm

- **Nghiệm ban đầu kém hiệu quả:** Nếu như ta chọn các Pattern khởi đầu kém hiệu quả, thì quá trình tìm kiếm nghiệm tối ưu của bài toán có thể lâu hơn mức dự kiến.
- **Bài toán phụ phức tạp:** Kết quả và hiệu suất của thuật toán phụ thuộc nhiều vào bài toán phụ và chiến lược chọn cột. Trong nhiều trường hợp, bài toán phụ có thể trở nên phức tạp hoặc thậm chí không thể mô hình hóa được, dẫn đến việc giải bài toán phụ có thể chiếm phần lớn thời gian, giảm hiệu suất của giải thuật và có thể không đạt được lời giải mong muốn trong một khoảng thời gian chấp nhận được.

4.2 Giải thuật tham lam kết hợp Simulated Annealing

Nói đến các phương pháp Heuristic là nói đến các phương pháp giải quyết các bài toán dựa trên việc sử dụng các quy tắc và kinh nghiệm thực tế thay vì sử dụng các lý thuyết chính xác. Được sử dụng rộng rãi trong các lĩnh vực như trí tuệ nhân tạo, khoa học máy tính, và tâm lý học, các phương pháp Heuristic giúp quá trình tính toán và đưa ra các quyết định dễ dàng và nhanh chóng hơn, đồng thời cũng tốn ít tài nguyên hơn so với các phương pháp chính xác truyền thống. Mặc dù các phương pháp Heuristic thường không đảm bảo tìm ra giải pháp tốt nhất, nhưng nó lại được ưa chuộng do dễ thích nghi và có thể áp dụng cho nhiều loại vấn đề khác nhau mà không cần phải tinh chỉnh quá nhiều. Ở phần dưới nhóm sẽ trình bày chi tiết về giải thuật tham lam kết hợp với Simulated Annealing, một trong những giải thuật Heuristic tương dối đơn giản.

4.2.1 Giải thuật tham lam (Greedy Algorithm)

Có thể nói giải thuật tham lam là một trong những giải thuật đơn giản nhất, thời gian tính toán của nó thường nhanh hơn rất nhiều so với các giải thuật khác, do ta không cần phải "suy nghĩ" khi chọn lựa các nước đi thích hợp. Giả sử trong bài toán 2D CSP của ta, kích thước của mẫu vật liệu có thể là giống nhau hoặc khác nhau, nếu các mẫu vật liệu có kích thước khác nhau, ta chỉ cần sắp xếp chúng theo chiều giảm dần của 1 tiêu chí nào đó, ở đây thích hợp nhất sẽ là chiều giảm dần diện tích của các mẫu vật liệu. Ta muốn cắt ra n loại sản phẩm, mỗi loại sẽ có chiều rộng, chiều cao và số lượng yêu cầu tối thiểu lần lượt là w_i, h_i và d_i, với $i = 1, 2, ..., n$. Đầu tiên, ta cũng sẽ sắp xếp n loại sản phẩm này theo thứ tự giảm dần diện tích, tức $w_1 \cdot h_1 \geq w_2 \cdot h_2 \geq ... \geq w_n \cdot h_n$. Sau đó lần lượt cắt các sản phẩm này từ các mẫu vật liệu theo đúng thứ tự đã sắp xếp, theo một quy tắc cắt nào đó để tối đa hóa khả năng tận dụng không gian trống, ở đây ta sẽ sử dụng quy tắc Bottom-Left, tức sẽ chọn vị trí thấp nhất và bên trái nhất để cắt sản phẩm, ưu tiên cắt mẫu vật liệu có diện tích lớn nhất vẫn còn đủ không gian để chứa sản phẩm này (Figure 9).

Ngoài ra, ở bước cuối, ta còn thực hiện thêm cơ chế làm khít (Tighten) để tối thiểu hóa lượng vật liệu thừa, bằng cách lặp qua tất cả các mẫu vật liệu đã sử dụng, theo thứ tự từ mẫu vật liệu có diện tích thừa lớn nhất tới nhỏ nhất, và đưa tất cả các sản phẩm bên trong nó đặt vào mẫu vật liệu trống khác (nếu tồn tại) có diện tích nhỏ nhất và đủ chứa tất cả sản phẩm của mẫu vật liệu ban đầu.

Nhìn chung, giải thuật tham lam tương đối dễ triển khai do không đòi hỏi ta phải mô hình hóa bài toán một cách phức tạp. Tuy nhiên như đã nói, giải thuật tham lam sẽ không đảm bảo

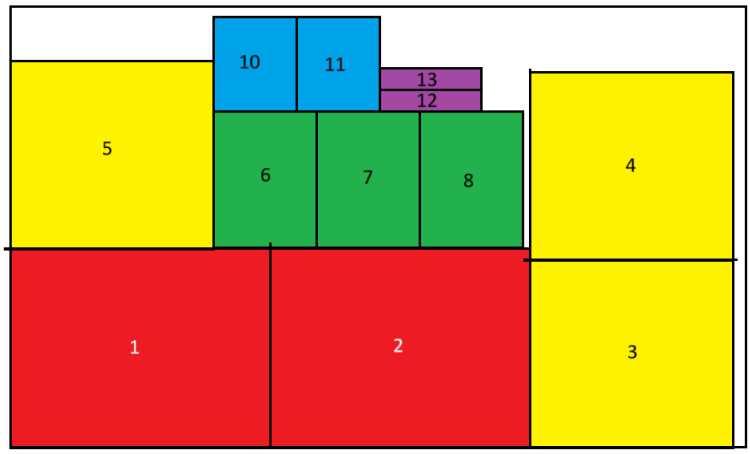


Figure 9: *Thứ tự cắt các sản phẩm theo giải thuật tham lam (sản phẩm có diện tích lớn nhất cắt trước, cắt theo quy tắc Bottom-Left).*

tìm ra lời giải tối ưu toàn cục, mà thay vào đó nó có thể dẫn đến lời giải cục bộ do chỉ tập trung vào lựa chọn phương án tối ưu nhất thời điểm hiện tại. Mã giả của giải thuật tham lam được cho ở bên dưới (algorithm 2).

4.2.2 Giải thuật Simulated Annealing (SA)

Simulated Annealing (SA) là một trong những thuật toán tối ưu hóa thuộc loại Heuristic, được xây dựng dựa trên cơ chế "annealing" trong quá trình luyện kim, một kĩ thuật liên quan đến việc kiểm soát nhiệt độ để thay đổi hình dạng vật lý của vật liệu. Về cơ bản, Simulated Annealing khá giống với Reinforcement Learning ở việc tại những thời điểm ban đầu, nó sẽ chú trọng đến quá trình khám phá (Exploration) hơn thay vì chú trọng quá trình khai thác (Exploitation). Nhưng sau khi quá trình tìm nghiệm diễn ra được một thời gian, nó sẽ chú trọng hơn vào việc khai thác, tức cẩn thận chọn lựa các nước đi tối ưu hơn thay vì chọn các nước đi ngẫu nhiên để khám phá miền nghiệm.

Về ý tưởng, ban đầu ta sẽ chọn 1 State (trạng thái) bất kì của hệ thống làm điểm xuất phát. Trong bài toán 2D CSP, thì State ban đầu là 1 cách cắt hợp lệ đáp ứng đủ số lượng các sản phẩm yêu cầu. Giả sử ta muốn cắt ra n loại sản phẩm, mỗi loại sẽ có chiều rộng, chiều cao và số lượng yêu cầu tối thiểu lần lượt là w_i, h_i và d_i, với $i = 1, 2, ..., n$, gọi n' là tổng số sản phẩm cần cắt, tức $n' = \sum_{i=1}^{n} d_i$, mỗi sản phẩm thứ i trong số n' sản phẩm này ($i = 1, 2, ..., n'$) sẽ được đặc trưng bởi 5-Tuple $(w'_i, h'_i, z_i, x_i, y_i)$, lần lượt là chiều rộng, chiều cao, Index của mẫu vật liệu nó thuộc về (Index bắt đầu từ 1 tới m, kích thước các mẫu vật liệu có thể khác nhau, chiều rộng và chiều cao lần lượt là W_i và H_i, $1 \leq i \leq m$), hoành độ và tung độ của góc trái dưới của sản phẩm này so với góc trái dưới của mẫu vật liệu chứa nó. Tạm gọi 5-Tuple này là Vector đặc trưng của mỗi sản phẩm, như vậy 1 State của hệ thống sẽ bao gồm Vector đặc trưng của tất cả các sản phẩm, ta có thể biểu diễn State này thành 1 ma trận S, mỗi hàng là Vector đặc trưng của 1 sản phẩm:

Input: Kích thước mẫu vật liệu, danh sách kích thước và yêu cầu tối thiểu của mỗi sản phẩm.

Output: Danh sách vị trí của mỗi sản phẩm kèm theo Index của mẫu vật liệu nó thuộc về.

1. Khởi tạo tập $S = \{\}$ dùng để lưu danh sách vị trí của các sản phẩm.

2. Khởi tạo tập $R = \{\}$ dùng để lưu Index của mẫu vật liệu mỗi sản phẩm thuộc về.

3. Sắp xếp lại các mẫu vật liệu và sản phẩm theo chiều giảm dần diện tích.

4. $n \leftarrow$ tổng số lượng sản phẩm cần cắt.

5. **while** $|S| < n$:

- **5.1.** Chọn vị trí x ở bên dưới cùng và trái nhất của mẫu vật liệu có diện tích lớn nhất vẫn còn đủ chỗ trống để cắt sản phẩm tiếp theo.
- **5.2.** Thêm x vào S, và Index của mẫu vật liệu được dùng vào R.

6. Thực hiện việc làm khít trên S và R.

7. Trả về S, R.

Algorithm 2: Greedy Algorithm

$$S = \begin{pmatrix} w'_1 & h'_1 & z_1 & x_1 & y_1 \\ w'_2 & h'_2 & z_2 & x_2 & y_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ w'_{n'} & h'_{n'} & z_{n'} & x_{n'} & y_{n'} \end{pmatrix} \tag{62}$$

Ở đây, ta sẽ dùng giải thuật tham lam đã trình bày ở phần trước để khởi tạo giá trị cho S. Nghĩa là, mục đích chính của việc sử dụng Simulated Annealing là để tinh chỉnh lại kết quả từ giải thuật tham lam, với mong muốn tìm được kết quả tối ưu hơn. Ý tưởng chủ đạo của Simulated Annealing là sau khi ta đã chọn được 1 State hợp lệ, ta sẽ bắt đầu "khám phá" từ State này. Ta sẽ phải định nghĩa một hàm chuyển trạng thái $f(S)$, có tác dụng trả về State S' mới dựa trên State S hiện tại, thông thường là một vị trí ngẫu nhiên nằm gần S. Tiếp theo, ta cần định nghĩa một hàm năng lượng (Energy Function), kí hiệu $E(S)$, có tác dụng trả về một số thực dựa trên State S hiện tại. Số thực này có thể được coi là mức năng lượng của toàn hệ thống, và mục tiêu của ta là tối thiểu nó. Trong bối cảnh của bài toán 2D CSP, ta sẽ định nghĩa hàm năng lượng như sau dựa trên ý tưởng của Dagli và Hajakbari [**Dagli**]:

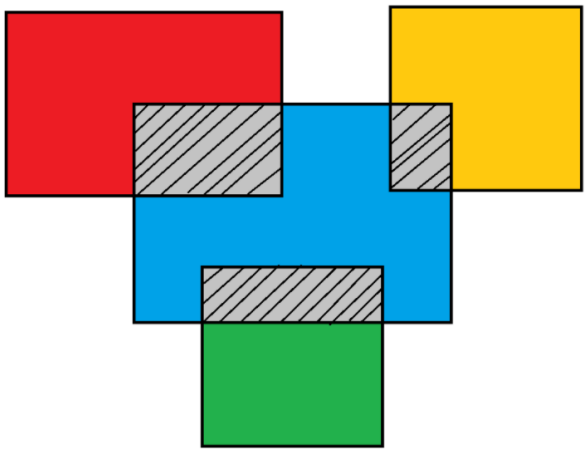


Figure 10: *Minh họa phần diện tích bị đè lên nhau của các sản phẩm (phần gạch sọc).*

$$E(S) = \lambda_1 \cdot A + \lambda_2 \cdot N + \lambda_3 \cdot D, \tag{63}$$

$$\text{with} \quad N = \sum_{j \in \{z_i | 1 \le i \le n'\}} W_j \cdot H_j, \tag{64}$$

$$D = \sum_{\substack{i,j=1,2,\ldots,n' \\ i<j \\ z_i=z_j}} \left((x_i + \frac{w'_i}{2} - x_j - \frac{w'_j}{2})^2 + (y_i + \frac{h'_i}{2} - y_j - \frac{h'_j}{2})^2 \right) \tag{65}$$

$$A = \sum_{\substack{i,j=1,2,\ldots,n' \\ i<j \\ z_i=z_j}} \Delta w'_{ij} \cdot \Delta h'_{ij} \tag{66}$$

$$\Delta w'_{ij} = \max(0, \min(x_i + w'_i, x_j + w'_j) - \max(x_i, x_j)) \tag{67}$$

$$\Delta h'_{ij} = \max(0, \min(y_i + h'_i, y_j + h'_j) - \max(y_i, y_j)) \tag{68}$$

Ở đây, A bằng tổng diện tích bị đè lên nhau của từng cặp sản phẩm trong cùng 1 mẫu vật liệu (Figure 10), N là tổng diện tích các mẫu vật liệu đã dùng và D là tổng bình phương khoảng cách giữa các cặp sản phẩm trong cùng 1 mẫu vật liệu, để đơn giản, ta quy khoảng cách giữa 2 sản phẩm là khoảng cách giữa 2 tâm của chúng. λ_1, λ_2 và λ_3 là các hệ số dương. Như vậy, khi ta cố gắng làm giảm $E(S)$, thì cũng đồng nghĩa với việc ta làm giảm phần diện tích A đè lên nhau của các sản phẩm, làm giảm tổng bình phương khoảng cách D giữa các sản phẩm nằm trong cùng 1 mẫu vật liệu, và cũng đồng thời làm giảm tổng diện tích các mẫu vật liệu sử dụng N. Nghĩa là ta muốn các sản phẩm phải được xếp sát nhau nhưng không bị đè lên nhau, và tận dụng tối đa phần không gian trống của các mẫu vật liệu. Như vậy việc lựa chọn giá trị cho các trọng số λ_1, λ_2 và λ_3 là hết sức quan trọng trong giải thuật Simulated Annealing, thông thường ta sẽ chọn λ_1 rất lớn, bởi vì mục đích tối cao là không được làm các sản phẩm bị đè lên nhau.

Tiếp theo, ta cần xác định quy trình làm lạnh (Cooling Schedule), nghĩa là cần định nghĩa một nhiệt độ ban đầu T và quá trình suy giảm của nó. Thông thường, ta sẽ định nghĩa hệ số làm lạnh (Cooling Rate) $\epsilon \in (0, 1)$, và cứ mỗi bước của thuật toán ta sẽ gán lại $T \leftarrow \epsilon \cdot T$. Bản

chất, giá trị của T càng lớn thì tỉ lệ "khám phá" càng cao, ta sẽ chấp nhận những nước đi rủi ro để khám phá miền nghiệm, có cơ hội nhảy ra khỏi các điểm tối ưu cục bộ (Local Optimum) để phát hiện ra các điểm tối ưu hơn. Khi thời gian trôi đi, do T giảm dần, tỉ lệ "khai thác" do đó cũng tăng lên, khi này ta không còn chú trọng vào việc khám phá xung quanh nữa mà cẩn thận hơn trong việc chọn State tiếp theo, khi T đủ nhỏ thì cũng là lúc ta dừng thuật toán. Cụ thể, giả sử State hiện tại là S, ta tính được State tiếp theo là $S' = f(S)$, khi này ta sẽ có 1 trong 2 lựa chọn, hoặc là đứng yên tại chỗ, hoặc là nhảy tới State S'. Định nghĩa hàm xác suất $P(E(S), E(S'), T)$ là hàm sẽ trả về xác suất ta sẽ lựa chọn nhảy tới State S' từ State S tại nhiệt độ T. Việc lựa chọn hàm xác suất như thế nào cũng là việc hết sức quan trọng, tuy nhiên, để đơn giản, ta sẽ chọn hàm xác suất có dạng như sau theo Kirkpatrick et al. [**Kirkpatrick**]:

$$P(E(S), E(S'), T) = \begin{cases} 1 & \text{nếu } E(S') < E(S), \\ e^{\frac{E(S)-E(S')}{T}} & \text{nếu } E(S') \geq E(S) \end{cases} \tag{69}$$

Dễ thấy nếu mức năng lượng tại State S' nhỏ hơn State S, thì rõ ràng State S' tối ưu hơn, nên ta sẽ không ngần ngại nhảy tới nó. Ngược lại, nếu State S' có mức năng lượng cao hơn nhiều so với State S, thì dễ thấy biểu thức $e^{\frac{E(S)-E(S')}{T}}$ sẽ cho kết quả gần 0 vì số mũ có giá trị âm lớn, tức khi này xác suất ta chọn nhảy tới S' sẽ thấp do rủi ro cao. Đồng thời khi T giảm dần, thì số mũ của $e^{\frac{E(S)-E(S')}{T}}$ cũng sẽ tăng dần theo độ lớn, khiến cả biểu thức cũng tiến dần tới 0, tạo hiệu ứng khi nhiệt độ giảm dần thì tỉ lệ "khám phá" cũng giảm dần.

Phần còn lại bây giờ là làm thế nào để định nghĩa hàm $f(S)$ cho bài toán 2D CSP đang xét? Nhận thấy rằng thứ chúng ta cần thay đổi để tạo ra State S' mới là các Vector đặc trưng trong State S, một số hành động mà ta có thể thực hiện lên Vector đặc trưng này như sau:

- Hoán đổi giá trị giữa w'_i và h'_i, nó sẽ có hiệu ứng quay sản phẩm thứ i 1 góc 90 độ tại chỗ.
- Thay đổi giá trị của x_i và y_i thành giá trị mới gần chúng, nó sẽ có hiệu ứng tịnh tiến vị trí cắt sản phẩm thứ i, cần đảm bảo vị trí mới không vượt ra ngoài phạm vi của mẫu vật liệu.
- Thay đổi giá trị của z_i thành một số nguyên dương gần nó, có hiệu ứng dịch chuyển sản phẩm thứ i sang mẫu vật liệu khác.
- Chọn ngẫu nhiên 2 Vector đặc trưng thứ i và thứ j, $1 \leq i < j \leq n'$, tiến hành hoán đổi giá trị giữa w_i với w_j, và giữa h_i với h_j, nó sẽ có hiệu ứng hoán đổi vị trí của 2 sản phẩm tương ứng.

Như vậy trong hàm $f(S)$, ta chỉ cần hiện thực để nó thực hiện ngẫu nhiên các công việc trên với các Vector đặc trưng ngẫu nhiên được chọn trong S.

Phần mã giả của giải thuật Simulated Annealing kết hợp Greedy Algorithm để giải bài toán 2D CSP được cho ở bên dưới (algorithm 3).

4.2.3 Ưu và nhược điểm của giải thuật tham lam kết hợp Simulated Annealing

Giải thuật tham lam kết hợp Simulated Annealing đã thể hiện những điểm tích cực trong việc giải bài toán 2D CSP, nhất là trong những trường hợp ta cần một giải thuật đơn giản nhưng đủ hiệu quả để đạt được nghiệm tối ưu mong muốn hoặc gần tối ưu. Tuy nhiên, xét cho cùng vì là giải thuật Heuristic, nên nó cũng còn tồn tại một số điểm yếu nhất định. Dưới đây, chúng ta sẽ phân tích những ưu điểm và nhược điểm của giải thuật này.

Input: Kích thước các mẫu vật liệu, danh sách kích thước và yêu cầu tối thiểu của mỗi sản phẩm.

Output: State tối ưu.

1. Thực hiện giải thuật tham lam để khởi tạo State S ban đầu.

2. $S_{optimal} \leftarrow S$, $N \leftarrow$ tổng diện tích các mẫu vật liệu đã sử dụng trong State S.

3. Đặt nhiệt độ ban đầu T, nhiệt độ tối thiểu T_0 và hệ số làm lạnh ϵ.

4. Định nghĩa các hàm $f(S)$, $E(S)$ và $P(E(S), E(S'), T)$.

5. Trong khi $T > T_0$:

- **5.1.** Tạo State mới $S' = f(S)$.
- **5.2.** $N' \leftarrow$ tổng diện tích các mẫu vật liệu sử dụng trong State S'.
- **5.3.** **Nếu** $N' < N$ **thì** $S_{optimal} \leftarrow S'$, $N \leftarrow N'$.
- **5.4.** Tính xác suất để chọn State mới $p = P(E(S), E(S'), T)$.
- **5.5.** **Nếu** $p \geq random(0, 1)$ **thì** $S \leftarrow S'$.
- **5.6.** $T \leftarrow \epsilon \cdot T$.

6. Trả về $S_{optimal}$.

Algorithm 3: Greedy Algorithm With Simulated Annealing

Ưu điểm

- **Nhanh, dễ triển khai:** Vì giải thuật này là sự kết hợp của 2 phương pháp tương đối đơn giản là giải thuật tham lam và Simulated Annealing, nên việc hiện thực nó không phải là vấn đề. Đồng thời, vì ta đã cung cấp một State khởi đầu khá thuận lợi cho Simulated Annealing thông qua kết quả từ thuật toán tham lam, nên thời gian hội tụ cũng được giảm đáng kể.
- **Thích hợp với không gian nghiệm lớn:** Với đặc tính của Simulated Annealing là khả năng tìm kiếm rộng và dễ dàng thoát ra khỏi điểm tối ưu cục bộ, những bài toán có không gian nghiệm lớn sẽ không còn là trở ngại.
- **Linh hoạt và dễ dàng mở rộng:** Khi thuật toán không đạt được hiệu quả cao, ta chỉ cần tinh chỉnh lại một vài tham số như T, ϵ, ... hay các hàm năng lượng, hàm xác suất ... mà không cần thay đổi quá nhiều về cấu trúc thuật toán. Đồng thời cũng vì sự đơn giản trong cấu trúc nên nó có thể dễ dàng mở rộng cho những bài toán khác nhau trong thực tế.

Nhược điểm

- **Không đảm bảo tìm được tối ưu toàn cục:** Đơn giản vì đây là giải thuật Heuristic nên sẽ có trường hợp bị mắc kẹt tại điểm tối ưu cục bộ, tuy nhiên điều này không thường xuyên xảy ra do đặc tính của Simulated Annealing.
- **Nhạy cảm với các tham số:** Hiệu suất của thuật toán phụ thuộc chủ yếu vào các tham số như nhiệt độ, hệ số làm lạnh và các hàm như hàm năng lượng, hàm chuyển trạng thái, ..., nên nếu các tham số và hàm này không được xác định hợp lí, ví dụ như quá trình làm mát, tức làm giảm nhiệt độ quá chậm, thì có thể dẫn tới thuật toán kém hiệu quả.

4.3 Hướng tiếp cận bằng Reinforcement Learning

Các thuật toán học máy (Machine Learning) thường được phân thành 3 loại lớn: Supervised Learning (học có giám sát), Unsupervised Learning (học không giám sát) và Reinforcement Learning (học tăng cường). Nếu như Supervised Learning là học từ một tập các dữ liệu được gắn nhãn để từ đó suy ra mối quan hệ giữa đầu vào và đầu ra, thì Unsupervised Learning lại không được cung cấp các dữ liệu được gắn nhãn, nhiệm vụ của nó thường chủ yếu là tìm cách mô tả cấu trúc không gian của dữ liệu. Loại thứ 3 là Reinforcement Learning - phương pháp tập trung vào việc làm thế nào để cho một Agent (ví dụ như nhân vật trong Game) trong một Environment (môi trường) có thể đưa ra các hành động sao cho lấy được phần thưởng nhiều nhất có thể. Khác với học có giám sát, học tăng cường không có dữ liệu được gắn nhãn trước làm đầu vào và cũng không có đánh giá các hành động là đúng hay sai. Tại section này, nhóm sẽ đề xuất một mô hình Reinforcement Learning để giải quyết bài toán 2D CSP trong trường hợp các mẫu vật liệu có kích thước bất kì, dựa trên giải thuật Policy Gradient đơn giản nhất kết hợp với Entropy Regularization.

4.3.1 Sơ lược về Policy Gradient

Ý tưởng chính của Policy Gradient là cố gắng làm tăng xác suất của các hành động mang lại tổng phần thưởng kỳ vọng cao hơn và giảm xác suất của các hành động mang lại tổng phần thưởng kỳ vọng thấp hơn, cho đến khi đạt được Policy tối ưu. Giả sử ta sử dụng Stochastic Policy π_θ, trong đó θ là tham số của mạng thần kinh (Neural Network), hiểu đơn giản thì π_θ sẽ

có đầu vào là State (trạng thái) hiện tại của Agent, nó sẽ trả về Vector chứa xác suất của các hành động, và Agent sẽ chọn ngẫu nhiên một trong các hành động này để thực hiện dựa theo xác suất của chúng. Giả sử trong 1 Episode, Agent của ta thực hiện lần lượt các hành động là A_0, A_1, ..., A_m, và vì thực hiện các hành động này, nó trải qua các State lần lượt là S_0, S_1, ..., S_m, S_{m+1}, trong đó S_{m+1} là một State ứng với trạng thái kết thúc. Như vậy, ta có thể định nghĩa một chuỗi sự kiện $p = \{(S_0, A_0), (S_1, A_1), ..., (S_m, A_m), S_{m+1}\}$, tạm gọi tập hợp tất cả các chuỗi sự kiện có thể xảy ra là Γ. Gọi $P(p|\theta)$ là xác suất để chuỗi sự kiện p diễn ra trong 1 Episode với tham số mạng thần kinh hiện tại, ta có:

$$P(p|\theta) = P(S_0) \prod_{t=0}^{m} \pi_\theta(A_t|S_t) P(S_{t+1}|S_t, A_t) \tag{70}$$

Trong đó:

- $P(S_0)$ là xác suất để Agent ở State S_0 tại thời điểm bắt đầu.
- $\pi_\theta(A_t|S_t)$ là xác suất để Agent thực hiện hành động A_t tại State S_t theo Policy hiện tại.
- $P(S_{t+1}|S_t, A_t)$ là xác suất để sau khi Agent thực hiện hành động A_t tại State S_t, thì nó sẽ chuyển sang State S_{t+1}.

Bây giờ ta sẽ thử tìm Gradient của $P(p|\theta)$ theo θ, nghĩa là ta muốn biết được khi θ thay đổi, thì xác suất của chuỗi sự kiện p sẽ tăng giảm như thế nào, tuy nhiên, để đơn giản, ta sẽ đi tìm Gradient của $ln(P(p|\theta))$, dễ thấy:

$$\nabla_\theta \ln(P(p|\theta)) = \nabla_\theta \left(\ln(P(S_0)) + \sum_{t=0}^{m} (\ln(\pi_\theta(A_t|S_t)) + \ln(P(S_{t+1}|S_t, A_t))) \right) = \nabla_\theta \sum_{t=0}^{m} \ln(\pi_\theta(A_t|S_t)) \tag{71}$$

Gọi $R(p)$ là tổng phần thưởng nhận được khi Agent trải qua đúng các sự kiện trong chuỗi sự kiện p, gọi $J(\pi_\theta)$ là kỳ vọng của tổng phần thưởng Agent nhận được nếu nó đi theo Policy hiện tại. Tức cho Agent trải qua một số lượng rất lớn Episode, mỗi Episode nó sẽ đi theo 1 chuỗi sự kiện p nào đó, thực hiện các hành động trong chuỗi sự kiện này và nhận được tổng phần thưởng là $R(p)$, thì $J(\pi_\theta)$ sẽ chính là trung bình giá trị $R(p)$ trong tất cả các Episode. Ta có:

$$J(\pi_\theta) = \sum_{p \in \Gamma} P(p|\theta) R(p) \tag{72}$$

Mục tiêu của ta là làm tăng $J(\pi_\theta)$, tức tăng kỳ vọng của tổng phần thưởng, như vậy cần phải tính Gradient của $J(\pi_\theta)$ và cập nhật tham số của mạng thần kinh theo hướng Gradient đó. Ta có:

$$\nabla_\theta J(\pi_\theta) = \nabla_\theta \sum_{p \in \Gamma} P(p|\theta) R(p) = \sum_{p \in \Gamma} (\nabla_\theta P(p|\theta)) R(p) = \sum_{p \in \Gamma} (P(p|\theta) \nabla_\theta ln(P(p|\theta))) R(p) \tag{73}$$

Thế kết quả từ biểu thức (71) vào biểu thức trên (ở đây thay m thành m_p, vì mỗi p khác nhau ta có m khác nhau), ta được:

$$\nabla_\theta J(\pi_\theta) = \sum_{p \in \Gamma} (P(p|\theta) \nabla_\theta \sum_{t=0}^{m_p} \ln(\pi_\theta(A_t|S_t))) R(p) = \mathop{\mathbb{E}}_{p \sim \pi_\theta} [R(p) \nabla_\theta \sum_{t=0}^{m_p} \ln(\pi_\theta(A_t|S_t))] \tag{74}$$

Như vậy từ biểu thức trên, ta có thể ước lượng được Gradient của $J(\pi_\theta)$ thông qua việc cho Agent trải qua khoảng N Episode, ta sẽ thu được tập N chuỗi sự kiện là $\Gamma = \{p_1, p_2, ..., p_N\}$. Như vậy Gradient ước tính sẽ là:

$$\hat{g} = \frac{1}{N} \sum_{p \in \Gamma} R(p) \nabla_\theta \sum_{t=0}^{m_p} \ln(\pi_\theta(A_t|S_t)) \tag{75}$$

Sau khi tính được Gradient $\hat{g}$, ta sẽ sử dụng Gradient Ascent để cập nhật tham số cho mạng thần kinh, ở đây nhóm chọn Adam Optimizer để thực hiện quá trình cập nhật.

4.3.2 Mô tả State Space, Action Space và phần thưởng

Giả sử bài toán 2D CSP của ta có m tấm vật liệu với kích thước khác nhau (tối đa m_{max} tấm vật liệu), yêu cầu cắt ra n loại sản phẩm (tối đa n_{max} loại sản phẩm), mỗi loại sản phẩm có số lượng yêu cầu là d_i, $1 \leq i \leq n$, số lượng yêu cầu của mỗi loại sẽ không quá d_{max}. Gọi chiều rộng và chiều cao của mỗi tấm vật liệu là W_i và H_i, $1 \leq i \leq m$, chiều rộng và chiều cao của mỗi loại sản phẩm là w_i và h_i, $1 \leq i \leq n$. Cách cắt sẽ là cắt ở vị trí nào cũng được, được phép quay sản phẩm 90 độ. Như vậy State Vector đầu vào của mạng thần kinh sẽ có dạng như Figure 11. Trong đó, các phần tử W_i và H_i với $m < i \leq m_{max}$ sẽ quy định bằng 0 vì chúng không tồn tại, tương tự các phần tử w_i và h_i với $n < i \leq n_{max}$ cũng sẽ quy định bằng 0. $x_{i,j}$ và $y_{i,j}$ lần lượt là hoành độ và tung độ của góc trái dưới của sản phẩm thứ j thuộc loại i trong mẫu vật liệu chứa nó (gốc tọa độ là góc trái dưới mẫu vật liệu), $1 \leq i \leq n_{max}$, $1 \leq j \leq d_{max}$, mặc định ban đầu chúng đều bằng 0. $z_{i,j}$ là Index của mẫu vật liệu đang chứa sản phẩm thứ j thuộc loại i, $1 \leq i \leq n_{max}$, $1 \leq j \leq d_{max}$, Index bắt đầu từ 1, kết thúc là m, nếu sản phẩm này không tồn tại, thì giá trị $z_{i,j}$ sẽ âm để biểu thị cho sự không tồn tại, nếu sản phẩm này chưa được cắt ra, thì $z_{i,j}$ sẽ $= 0$. Cuối cùng là $r_{i,j}$, có giá trị $= 1$ nếu như sản phẩm thứ j thuộc loại i bị quay 90 độ, ngược lại $=$ 0. Như vậy tổng cộng State Vector của ta sẽ có $2(m_{max} + n_{max}) + 4n_{max}d_{max}$ phần tử.

Trước khi đưa State Vector vào mạng thần kinh, ta cần chuẩn hóa nó để giá trị của các phần tử không quá lớn, thông thường ta sẽ chia các thành phần của State Vector cho các hằng số đã định trước để giá trị của các phần tử rơi vào đâu đó trong khoảng từ -1 đến 1, hoặc 0 đến 1. Sau khi chuẩn hóa State Vector, ta sẽ đưa nó vào mạng thần kinh, ở đây nhóm đã xây dựng 3 mạng thần kinh độc lập tương đối đơn giản. Mạng thần kinh thứ nhất sẽ nhận vào State Vector đã chuẩn hóa, cho State Vector này đi qua các lớp tuyến tính xen kẽ hàm kích hoạt Leaky RELU (tránh Vanishing Gradient), và trả về Vector xác suất dùng để dự đoán mẫu vật liệu tiếp theo sẽ cắt, như vậy kích thước Vector này là m_{max}. Mạng thần kinh thứ hai tương tự mạng thần kinh thứ nhất, chỉ có điều Vector xác suất của nó là để dự đoán loại sản phẩm tiếp theo sẽ cắt, như vậy kích thước của nó là n_{max}. Mạng thần kinh thứ 3 cũng tương tự, nhưng lớp cuối cùng chỉ bao gồm 1 Neuron được kích hoạt bởi hàm Sigmoid, nó sẽ có giá trị từ 0 tới 1 biểu thị cho xác suất sản phẩm tiếp theo sẽ bị quay (1 ứng với chắc chắn bị quay, 0 ứng với chắc chắn không quay).

Khi bắt đầu 1 Episode (chưa cắt mẫu vật liệu nào), phần thưởng có giá trị khởi tạo là 0. Sau khi đã chọn được mẫu vật liệu và loại sản phẩm để cắt dựa theo Vector xác suất, ta sẽ tiến hành kiểm tra xem loại sản phẩm thứ i đó có tồn tại hay không, nếu không, tức $n < i \leq n_{max}$, thì phần thưởng sẽ bị trừ đi 1, đồng thời nếu loại sản phẩm này đã được cắt hết, thì phần thưởng cũng sẽ bị trừ đi 1, sau đó cần tiến hành chọn lại loại sản phẩm cần cắt. Cơ chế chọn lại sẽ là chọn loại sản phẩm có diện tích lớn nhất đến nhỏ nhất. Tương tự, nếu mẫu vật liệu được chọn đã đầy, tức không thể cắt sản phẩm được chọn ra từ mẫu vật liệu này, thì phần thưởng cũng sẽ bị trừ đi 1, và tiến hành chọn mẫu vật liệu khác thay thế, quy tắc chọn lại cũng là chọn mẫu vật liệu có diện tích lớn nhất đến nhỏ nhất. Trong quá trình chọn lại mẫu vật liệu, ta cũng sẽ kết

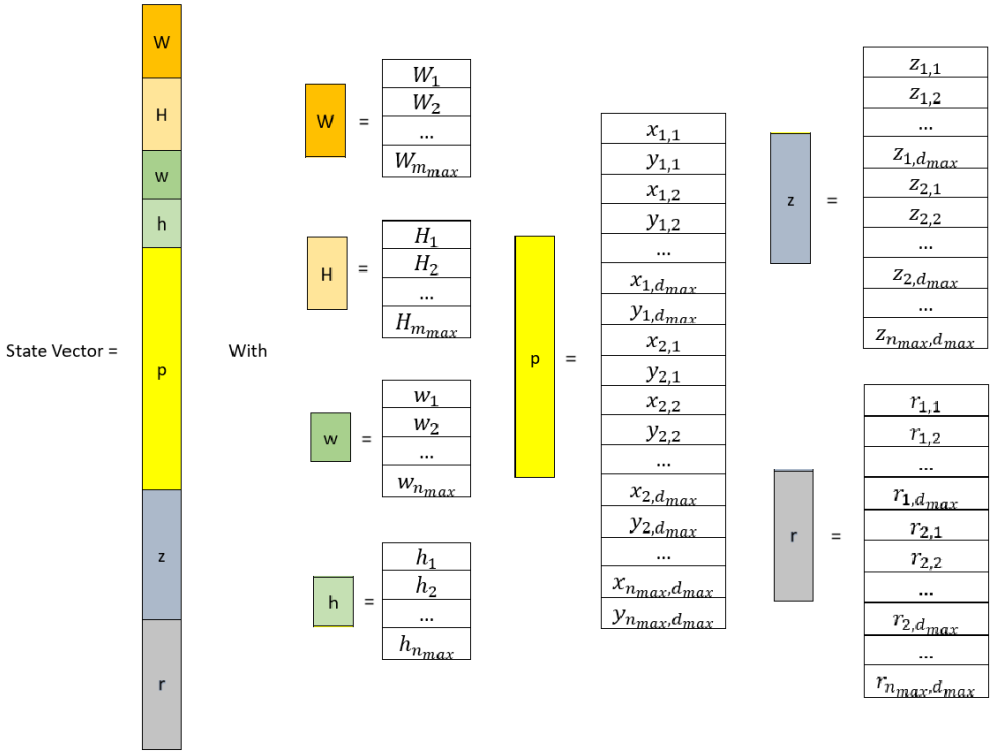


Figure 11: *Mô tả các thành phần của State Vector.*

hợp thử quay sản phẩm 90 độ xem nó có đặt vừa vào mẫu vật liệu hay không, mỗi lần chỉnh lại cách quay phần thưởng cũng bị trừ đi 1. Sau khi đã chọn được mẫu vật liệu, sản phẩm và cách quay thích hợp, ta sẽ tiến hành cắt sản phẩm theo quy tắc Bottom-Left như đã có nói ở phần giải thuật tham lam, đồng thời phần thưởng được cộng thêm 1 nếu mẫu vật liệu này đã bị cắt, ngược lại nếu là mẫu vật liệu mới, thì phần thưởng sẽ bị trừ đi một lượng tỉ lệ thuận với diện tích của mẫu vật liệu, tất cả điều này sẽ ép Agent phải lựa chọn cách cắt để tối ưu hóa tổng diện tích các mẫu vật liệu sử dụng nhất và tránh động tác thừa nhất. Sau khi cắt, ta sẽ lưu lại State Vector hiện tại và hành động mới thực hiện để Train, đồng thời cũng cập nhật lại State Vector cho lần chọn hành động tiếp theo. Lưu ý ở đây, do ta xuất ra 2 Vector xác suất chọn mẫu vật liệu và sản phẩm, thêm 1 giá trị xác suất để xác định xem có quay sản phẩm hay không, tức là 1 hành động của ta sẽ bao gồm tổ hợp của 3 hành động nhỏ, do đó giá trị $\pi_\theta(A_t|S_t)$ sẽ bằng tích xác suất của hành động chọn mẫu vật liệu nhân với xác suất của hành động chọn loại sản phẩm nhân với xác suất của hành động quay. Khi Test, ta sẽ không chọn ngẫu nhiên mẫu vật liệu và sản phẩm cần cắt theo xác suất của chúng, mà sẽ chọn mẫu vật liệu và sản phẩm có xác suất lớn nhất, và nếu xác suất quay > 0.5 thì ta sẽ quay, còn < 0.5 thì sẽ không quay, đồng thời ở giai đoạn cuối cùng, nhóm còn thực hiện thêm cơ chế "làm khít" đã trình bày ở phần giải thuật tham lam để tối thiểu hóa lượng vật liệu thừa.

4.3.3 Entropy Regularization

Ta có hàm Loss khi Train mô hình Reinforcement Learning đã trình bày như sau:

$$L_0 = -J(\pi_\theta) \tag{76}$$

Như vậy làm giảm giá trị Loss ở trên cũng đồng nghĩa làm tăng tổng phần thưởng kỳ vọng, tuy nhiên, đa số các hàm Loss lại có nhiều điểm tối ưu cục bộ, dẫn tới khi sử dụng Gradient Descent để cập nhật tham số, ta dễ dàng "mắc kẹt" tại vị trí của chúng, gây ra hiện tượng Agent chỉ chọn hoàn toàn 1 hành động duy nhất, bỏ qua việc xem xét các hành động có tiềm năng khác. Để tránh tình trạng này, nhóm đã trừ vào hàm Loss ở trên một lượng Entropy của mô hình. Entropy đặc trưng cho sự hỗn loạn, cụ thể ở đây là sự hỗn loạn trong việc chọn các hành động, như vậy khi ta trừ đi một lượng Entropy vào hàm Loss, thì khi tối ưu hàm Loss, nó sẽ có khuynh hướng làm tăng Entropy để giảm giá trị hàm Loss, nói cách khác, nó sẽ không cho phép hành động nào có xác suất được chọn là quá vượt trội, vì khi này Entropy sẽ thấp. Tóm lại, Entropy của mô hình sẽ được tính bằng công thức sau:

$$E = \lambda_1 E_1 + \lambda_2 E_2 + \lambda_3 E_3 \tag{77}$$

Trong đó:

- E_1 là Entropy của Vector xác suất chọn mẫu vật liệu tiếp theo sẽ cắt.
- E_2 là Entropy của Vector xác suất chọn sản phẩm tiếp theo để cắt.
- E_3 là Entropy của Vector xác suất xoay sản phẩm.
- λ_1, λ_2 và λ_3 là các trọng số dương.

Giả sử V là một Vector xác suất có n phần tử, phần tử thứ i kí hiệu là V_i, công thức để tính Entropy của Vector xác suất V như sau,:

$$E_V = -\sum_{i=1}^{n} V_i ln(V_i) \tag{78}$$

Dễ thấy Entropy luôn dương, nó sẽ tiến dần tới 0 nếu phân phối xác suất càng không đều, và sẽ lớn nhất khi phân phối xác suất là một phân phối đều (độ hỗn loạn cao nhất). Như vậy, tổng hợp lại, ta sẽ có hàm Loss sau:

$$L = L_0 - E \tag{79}$$

Kết hợp với kết quả từ (75), để tối ưu hàm Loss trên, ta có thể tối ưu biểu thức thay thế sau (giả sử ta đã trải qua N Episode):

$$L' = -\frac{1}{N}\sum_{p\in\Gamma} R(p)\sum_{t=0}^{m_p}(\ln(\pi_\theta(A_t^{stock}|S_t)) + \ln(\pi_\theta(A_t^{product}|S_t)) + \ln(\pi_\theta(A_t^{rotate}|S_t))) - E_N \tag{80}$$

Trong đó:

- A_t^{stock} là hành động chọn mẫu vật liệu tại thời điểm t.
- $A_t^{product}$ là hành động chọn loại sản phẩm tại thời điểm t.
- A_t^{rotate} là hành động quay sản phẩm tại thời điểm t.
- E_N là trung bình Entropy của mô hình trong tất cả các lần xuất ra các Vector xác suất để chọn hành động trong N Episode.

Như vậy, sau khi tính được được Gradient của L' theo tham số θ của mô hình, ta chỉ cần sử dụng Gradient Descent để tiến hành cập nhật nó.

4.3.4 Ưu và nhược điểm của Reinforcement Learning

Reinforcement Learning, không chỉ là công cụ đắc lực trong việc giải các bài toán CSP mà còn được ứng dụng rộng rãi trong vô số lĩnh vực khác, đã khẳng định ưu thế vượt trội của mình trong việc giải quyết các bài toán đầy thách thức. Tuy vậy, phương pháp này vẫn đối mặt với một số hạn chế đáng kể, chẳng hạn như thời gian Train kéo dài hay yêu cầu lượng tài nguyên tính toán và lưu trữ lớn, ... Dưới đây ta sẽ phân tích các điểm mạnh yếu của giải thuật này trong việc giải quyết bài toán 2D CSP:

Ưu điểm

- **Khả năng học hỏi từ kinh nghiệm:** Reinforcement Learning có thể học và tối ưu hóa chiến lược cắt thông qua quá trình tương tác nhiều lần với môi trường, nó sẽ tự tối ưu hóa các hành động mà không yêu cầu ta phải cung cấp các quy tắc quá phức tạp.
- **Hiệu quả với các bài toán lớn:** Khi State Space lớn, ta có thể dựa vào Deep Neural Network để đưa ra quyết định hành động.
- **Khả năng kết hợp:** Reinforcement Learning có thể được kết hợp với các giải thuật tối ưu khác một cách dễ dàng, điển hình như giải thuật tham lam để tối đa hóa hiệu suất.
- **Khả năng khám phá:** Nhờ cơ chế khám phá ngẫu nhiên, Agent có thể thử nghiệm các chiến lược chưa được xem xét trước đó, mở rộng phạm vi tìm kiếm và tăng khả năng tìm ra các giải pháp đột phá.
- **Dễ dàng thích ứng với sự thay đổi:** Khi môi trường thay đổi như số lượng mẫu vật liệu thay đổi, số lượng loại sản phẩm thay đổi, ... Ta chỉ cần chỉnh lại một vài siêu tham số của mô hình mà không cần xây dựng nó lại từ đầu.

Nhược điểm

- **Thời gian Train:** Reinforcement Learning yêu cầu rất nhiều vòng lặp để Agent học được chiến lược tối ưu. Với bài toán 2D CSP, State Space và Action Space lớn làm tăng đáng kể thời gian Train, chưa kể đến thời gian tương tác giữa Agent với môi trường.
- **Tài nguyên tính toán:** Vì mô hình Reinforcement Learning thường đi kèm với 1 Deep Neural Network, nên cần một lượng tài nguyên phần cứng mạnh mẽ như GPU hoặc TPU để xử lý.
- **Dễ mắc kẹt tại điểm tối ưu cục bộ:** Điều này thường xảy ra khi mô hình Reinforcement Learning không có cơ chế cân bằng giữa Exploration và Exploitation, dẫn đến mô hình quá chú trọng đến các hành động có lợi trước mắt.
- **Thiết kế phần thưởng khó khăn:** Muốn mô hình Reinforcement Learning đạt hiểu quả, việc tạo cơ chế phần thưởng hợp lý là hết sức quan trọng. Trong nhiều trường hợp, việc tín hiệu phần thưởng quá thưa thớt (Sparse Reward) có thể làm giảm hiệu quả Train mô hình.
- **Hiệu suất thấp với môi trường lạ:** Mô hình Reinforcement Learning thường có hiệu suất cao trong môi trường đã được Train, nhưng có thể sẽ không thích nghi hiệu quả với các trường hợp mới với dữ liệu đầu vào khác biệt.

5 Đánh giá hiệu suất

Ở phần này, nhóm sẽ đánh giá hiệu suất của các giải thuật đã trình bày, trong đó giải thuật tham lam kết hợp Simulated Annealing và giải thuật Reinforcement Learning sẽ được chạy trong môi trường gymnasium với một bộ dữ liệu ngẫu nhiên, còn giải thuật Column Generation và phương pháp Simplex được thực hiện trên công cụ mà nhóm đã tạo ra bằng ngôn ngữ lập trình Python. Mã nguồn của phần mềm này nằm trong thư mục "2D CSP Solver". Chi tiết về cách sử dụng phần mềm sẽ được trình bày trong subsection 6.1.

5.1 Đánh giá hiệu suất của phương pháp Simplex

Đầu tiên nhóm sẽ đánh giá mô hình thứ nhất, ứng với trường hợp các mẫu vật liệu có kích thước cố định. Bảng bên dưới là kết quả chạy thử mô hình này với 4 Test Case trên phần mềm của nhóm:

Problem No.	Stock size (W, H)	No. stocks	Products (w, h, d)	No. stocks used	Total area	Waste area	Waste rate	Average time (s)
1	(100, 100)	3	(40, 50, 3) (25, 40, 2)	1	15000	2000	0.2	0.2157
2	(100, 200)	4	(50, 150, 3) (25, 40, 2) (20, 10, 3)	2	40000	14900	0.3725	0.5526
3	(150, 200)	6	(90, 50, 2) (80, 100, 3) (50, 60, 2)	2	60000	21000	0.35	7.775
4	(123, 256)	8	(36, 78, 2) (45, 94, 3) (98, 76, 2) (100, 56, 1)	2	62976	24174	0.3839	152.48

Table 3: Kết quả đánh giá tốc độ của mô hình 1.

Dễ thấy ở cột "Average time", khi kích thước bài toán tăng lên, thì thời gian cần để giải mô hình quy hoạch tuyến tính nguyên này tăng lên rất nhanh. Do đó nó chỉ phù hợp khi ta muốn tìm nghiệm chính xác cho các bài toán nhỏ, khi số lượng sản phẩm cần cắt không vượt quá 10.

Đến với mô hình thứ 2, ứng với trường hợp các mẫu vật liệu có kích thước khác nhau, được dùng không giới hạn, và kiểu cắt thuộc loại 2-Stage Guillotine có xén. Bảng dưới là kết quả chạy thử mô hình này với 4 Test Case trên phần mềm của nhóm:

Problem No.	Stocks (W, H)	Products (w, h, d)	No. stocks used	Total area	Waste area	Waste rate	Average time (s)
1	(100, 200) (150, 90)	(20, 30, 3) (50, 70, 2) (90, 15, 2)	2	27000	15500	0.5741	0.1450
2	(140, 257) (128, 369)	(55, 66, 4) (57, 111, 3) (78, 89, 6)	3	107940	32787	0.3038	0.2513
3	(210, 213) (254, 222) (255, 213)	(50, 102, 4) (99, 12, 3) (68, 67, 6) (100, 58, 4)	2	99045	24545	0.2478	3.0367
4	(100, 150) (70, 80) (190, 160) (185, 185)	(50, 65, 5) (90, 40, 4) (60, 80, 7) (75, 75, 5) (95, 100, 5)	8	164450	24575	0.1494	12.9228

Table 4: Kết quả đánh giá tốc độ của mô hình 2.

Nhận thấy tốc độ của mô hình này hơn hẳn mô hình thứ nhất, điều này là do ở mô hình này, số lượng mẫu vật liệu được phép dùng là không giới hạn, đồng thời kiểu cắt cũng bị giới hạn thành 2-Stage Guillotine có xén, nên mô hình tương đối dễ để giải. Tuy nhiên, mô hình này cũng chỉ thích hợp để giải các bài toán nhỏ, vì khi tăng số lượng mẫu vật liệu cũng như số sản phẩm, thời gian tìm nghiệm sẽ tăng rất nhanh do số lượng biến nhị phân tăng lên đáng kể.

Đến với mô hình thứ 3, ứng với trường hợp các mẫu vật liệu có kích thước khác nhau và số lượng mẫu vật liệu là hữu hạn. Bảng dưới là kết quả chạy thử mô hình này với 4 Test Case trên phần mềm của nhóm:

Problem No.	Stocks (W, H, D)	Products (w, h, d)	No. stocks used	Total area	Waste area	Waste rate	Average time (s)
1	(100, 200, 1) (150, 90, 2)	(20, 30, 2) (50, 70, 2) (90, 15, 1)	1	13500	3950	0.2925	0.1044
2	(100, 200, 1) (150, 90, 2) (200, 300, 1)	(30, 30, 1) (50, 70, 3) (90, 15, 2) (90, 100, 1)	2	33500	10400	0.3104	14.0969
3	(100, 200, 3) (150, 90, 2) (30, 45, 1) (50, 80, 1)	(30, 30, 2) (50, 70, 2) (90, 15, 2)	1	13500	2000	0.1481	44.9055
4	(100, 200, 3) (150, 90, 2)	(30, 30, 2) (50, 70, 2) (90, 15, 2) (90, 100, 1)	2	33500	13000	0.3881	72.014

Table 5: Kết quả đánh giá tốc độ của mô hình 3.

Dễ thấy tốc độ để giải mô hình tuyến tính nguyên này khá chậm, chỉ với một vài sự thay đổi nhỏ trong các mẫu vật liệu hoặc sản phẩm cũng làm cho tốc độ giải bài toán tăng lên đáng kể. Chính vì vậy với trường hợp này, nhóm ưu tiên chọn các giải thuật Heuristic để giải bài toán.

5.2 Đánh giá hiệu suất của giải thuật Column Generation

Để đánh giá tốc độ của giải thuật này, nhóm đã thử chạy nó với trường hợp kích thước các mẫu vật liệu là cố định, bảng bên dưới là kết quả chạy thử giải thuật này trên phần mềm của nhóm với 4 Test Case tăng dần theo độ phức tạp:

Problem No.	Stocks (W, H)	Products (w, h, d)	No. stocks used	Total area	Waste area	Waste rate	Average time (s)
1	(100, 200)	(30, 30, 5) (50, 70, 6) (90, 15, 7)	3	60000	25050	0.4175	0.1778
2	(300, 250)	(50, 60, 10) (80, 70, 9) (100, 100, 10) (75, 90, 6)	4	300000	79100	0.2637	0.1148
3	(500, 800)	(70, 55, 16) (86, 99, 12) (150, 200, 15) (88, 97, 6) (200, 254, 13) (255, 400, 20)	11	4400000	1034616	0.2351	0.0790
4	(2000, 1800)	(100, 369, 30) (200, 155, 40) (400, 555, 50) (150, 200, 25) (500, 1000, 55) (788, 999, 10)	19	68400000	18830880	0.2753	0.1846

Table 6: Kết quả đánh giá tốc độ của giải thuật Column Generation.

Như vậy, giải thuật Column Generation là phương án khá tốt cho trường hợp kích thước các mẫu vật liệu là cố định và kiểu cắt bị giới hạn thành 2-Stage Guillotine có xén. Thời gian để giải bài toán không tăng đột ngột như giải thuật Simplex khi số lượng sản phẩm cần cắt tăng lên. Tuy nhiên, độ tối ưu của giải thuật này thường không cao, dễ thấy ở cột "Waste rate" tỉ lệ lãng phí vẫn còn khá lớn.

5.3 Đánh giá hiệu suất của giải thuật tham lam kết hợp Simulated Annealing

Để đánh giá độ hiệu quả của giải thuật, nhóm đã cho chạy thử với một vài Test Case trong môi trường gymnasium với số lượng mẫu vật liệu và số lượng sản phẩm tăng dần, chi tiết các Test Case sử dụng có thể được xem tại đây. Mỗi File dữ liệu sẽ có nội dung như sau:

- Dòng thứ nhất là dòng ghi chú "Stock:".
- Các dòng tiếp theo, mỗi dòng sẽ ứng với 1 mẫu vật liệu, có cú pháp <Chiều rộng>, <Chiều cao>.

- Dòng tiếp theo là dòng ghi chú "Products:".
- Các dòng tiếp theo, mỗi dòng sẽ ứng với 1 loại sản phẩm, có cú pháp <Chiều rộng>, <Chiều cao>, <Số lượng yêu cầu>.

Với mỗi Test Case, kết quả đầu ra sẽ được đánh giá dựa trên các thông số sau:

- Total area, là tổng diện tích của các mẫu vật liệu bị cắt.
- Waste area, là tổng diện tích thừa của các mẫu vật liệu bị cắt.
- Waste rate, là tỉ lệ lãng phí, bằng tổng diện tích thừa chia tổng diện tích các mẫu vật liệu bị cắt.
- No. trials, là số lần Simulated Annealing thử một cách cắt khác.
- No. improvements, là số lần Simulated Annealing thử một cách cắt khác và cách cắt này hiệu quả hơn cách cắt tốt nhất hiện tại.
- Improvement rate, là tỉ lệ cải thiện giải pháp bằng Simulated Annealing, có giá trị = No. improvements / No. trials.
- Average time, là thời gian trung bình để tìm ra giải pháp tối ưu sau 10 lần chạy thử với Test Case này, không tính độ trễ do môi trường gymnasium.

Sau khi chạy chương trình với các Test Case, kết quả thu được như sau:

Problem No.	No. stocks	No. products	Total area	Waste area	Waste rate	No. trials	No. improvements	Improvement rate	Average time (s)
1	36	30	9700	2922	0.3012	40	3	0.0750	0.2610
2	49	50	40632	4699	0.1156	51	4	0.0784	0.2868
3	64	60	42695	3255	0.0762	53	3	0.0566	0.4051
4	81	80	33991	2655	0.0781	53	4	0.0755	0.9166
5	100	100	40756	2599	0.0638	53	4	0.0755	1.2693
6	100	110	70956	5099	0.0719	54	5	0.0926	1.3711
7	100	120	119863	7599	0.0634	55	6	0.1091	1.2983
8	100	130	119888	11195	0.0934	55	5	0.0909	1.5832
9	100	140	65641	2593	0.0395	60	4	0.0667	2.1118
10	100	150	139197	10005	0.0719	68	7	0.1029	2.0095

Table 7: Kết quả đánh giá nội bộ của giải thuật tham lam kết hợp Simulated Annealing.

Nhận xét:

- Ở cột "Total area" và "Waste area", ta thấy tổng diện tích các mẫu vật liệu sử dụng và tổng diện tích vật liệu thừa không tăng nghiêm ngặt khi số sản phẩm cần cắt tăng lên, như vậy có nghĩa là khi kích thước bài toán tăng lên, thì chưa chắc lượng mẫu vật liệu cần sử dụng và sự lãng phí diện tích cũng sẽ tăng lên.
- Ở cột "Waste rate", tỉ lệ lãng phí diện tích cao nhất là ở Test Case đầu tiên, đây là Test Case có số lượng mẫu vật liệu và sản phẩm cần cắt là ít nhất, điều này có nhiều nguyên nhân, thường là do ta có ít sự lựa chọn trong việc chọn ra mẫu vật liệu phù hợp để cắt sản phẩm. Khi số lượng mẫu vật liệu và sản phẩm cần cắt tăng dần, ta thấy tỉ lệ lãng phí có xu hướng giảm xuống, do khi này ta có nhiều sự lựa chọn hơn trong việc chọn mẫu vật liệu phù hợp.

- Ở cột "No. trials" và cột "No. improvements", ta thấy số lượng lần thử giải pháp mới của Simulated Annealing và số lần giải pháp mới tối ưu hơn giải pháp cũ có xu hướng tăng lên, điều này là do sự tăng kích thước của không gian nghiệm.
- Cột "Improvement rate" cho thấy rằng, ở không gian nghiệm càng lớn, thì tỉ lệ cải thiện giải pháp cũ của Simulated Annealing có xu hướng tăng lên, điều này cho thấy tính ưu việt của nó khi dễ dàng thoát khỏi điểm tối ưu cục bộ của giải thuật tham lam.
- Cột "Average time" cho thấy thời gian chạy thuật toán là khá nhanh, có xu hướng tăng lên khi không gian nghiệm tăng.

Tóm lại, giải thuật tham lam kết hợp Simulated Annealing khá mạnh, vượt trội hơn hẳn so với các giải thuật Simplex và Column Generation trước đó, cả về phạm vi bài toán và tốc độ giải bài toán.

5.4 Đánh giá hiệu suất của mô hình Reinforcement Learning

Để đánh giá độ hiệu quả của giải thuật, nhóm đã cho chạy thử với một vài Test Case trong môi trường gymnasium với số lượng mẫu vật liệu và số lượng sản phẩm tăng dần, nhóm dùng lại bộ dữ liệu của phần trước để đánh giá và để tiện so sánh giữa hai giải thuật. Ở đây, số mẫu vật liệu tối đa là 100, kích thước chiều rộng lẫn chiều cao của chúng dao động trong khoảng từ 50 tới 100. Số loại sản phẩm cần cắt tối đa là 24 và yêu cầu mỗi loại tối đa là 20. Nghĩa là mô hình Reinforcement Learning của nhóm sẽ có State Vector với tổng cộng 2168 phần tử, Output của mô hình sẽ bao gồm Vector xác suất chọn mẫu vật liệu gồm 100 phần tử, Vector xác suất chọn loại sản phẩm gồm 24 phần tử, và cuối cùng là xác suất quay (1 phần tử).

5.4.1 Tổng quan cấu trúc mạng thần kinh

Trước khi đánh giá hiệu suất nhóm sẽ trình bày cấu trúc mạng thần kinh đã sử dụng. Bảng bên dưới là danh sách các lớp được sử dụng trong phần mạng thần kinh dùng để biến đổi State Vector thành Vector xác suất chọn mẫu vật liệu:

Layer	Input Size	Output Size	Number of Parameters	Activation
Linear 1	2168	1536	$3,331,584$	Leaky ReLU, $\alpha = 0.01$
Linear 2	1536	1024	$1,573,888$	Leaky ReLU, $\alpha = 0.01$
Linear 3	1024	512	$524,800$	Leaky ReLU, $\alpha = 0.01$
Linear 4	512	256	$131,328$	Leaky ReLU, $\alpha = 0.01$
Linear 5	256	128	$32,896$	Leaky ReLU, $\alpha = 0.01$
Linear 6	128	100	$12,900$	Leaky ReLU, $\alpha = 0.01$
Linear 7	100	100	$10,100$	Softmax

Table 8: Cấu trúc Stock Prediction Layers.

Bảng bên dưới là danh sách các lớp được sử dụng trong phần mạng thần kinh dùng để biến đổi State Vector thành Vector xác suất chọn loại sản phẩm:

Layer	Input Size	Output Size	Number of Parameters	Activation
Linear 8	2168	1536	$3,331,584$	Leaky ReLU, $\alpha = 0.01$
Linear 9	1536	1024	$1,573,888$	Leaky ReLU, $\alpha = 0.01$
Linear 10	1024	512	$524,800$	Leaky ReLU, $\alpha = 0.01$
Linear 11	512	256	$131,328$	Leaky ReLU, $\alpha = 0.01$
Linear 12	256	128	$32,896$	Leaky ReLU, $\alpha = 0.01$
Linear 13	128	64	$8,256$	Leaky ReLU, $\alpha = 0.01$
Linear 14	64	24	$1,560$	Softmax

Table 9: Cấu trúc Product Prediction Layers.

Bảng bên dưới là danh sách các lớp được sử dụng trong phần mạng thần kinh dùng để biến đổi State Vector thành xác suất quay sản phẩm:

Layer	Input Size	Output Size	Number of Parameters	Activation
Linear 15	2168	1024	$2,221,056$	Leaky ReLU, $\alpha = 0.01$
Linear 16	1024	512	$524,800$	Leaky ReLU, $\alpha = 0.01$
Linear 17	512	256	$131,328$	Leaky ReLU, $\alpha = 0.01$
Linear 18	256	128	$32,896$	Leaky ReLU, $\alpha = 0.01$
Linear 19	128	32	$4,128$	Leaky ReLU, $\alpha = 0.01$
Linear 20	32	1	33	Sigmoid

Table 10: Cấu trúc Rotation Prediction Layers.

Như vậy, mô hình này có tổng cộng 20 lớp tuyến tính, 17 lớp Leaky RELU với $\alpha = 0.01$, 2 lớp Softmax và 1 lớp Sigmoid. Tổng số lượng tham số là $14,136,049$. Kích thước mô hình vào cỡ 53.925 MB. Với số lượng tham số tương đối lớn, quá trình Train mô hình này tốn khoảng hơn 1 tiếng trên CPU. Như vậy thay vì xây dựng các mô hình phức tạp, nhóm đã quyết định xây dựng một mô hình đơn giản với nhiều lớp, vì tin rằng khi mô hình càng có nhiều lớp, giá trị tối ưu cục bộ sẽ không chênh lệch quá xa so với giá trị tối ưu toàn cục, khi đó nếu Agent bị mắc kẹt tại điểm tối ưu cục bộ, thì kết quả vẫn có thể chấp nhận được.

5.4.2 Đánh giá hiệu suất với một bộ dữ liệu

Sau khi chạy thử mô hình với các Test Case ở phần trước (xem tại đây) trong môi trường gymnasium, kết quả thu được như sau (Table 11):

Nhận xét:

- Ở cột "Total area" và "Waste area", ta cũng thấy tổng diện tích các mẫu vật liệu sử dụng và tổng diện tích vật liệu thừa không tăng nghiêm ngặt khi số sản phẩm cần cắt tăng lên, như vậy có nghĩa là khi kích thước bài toán tăng lên, thì chưa chắc lượng mẫu vật liệu cần sử dụng và sự lãng phí diện tích cũng sẽ tăng lên.

- Ở cột "Waste rate", tỉ lệ lãng phí diện tích cao nhất vẫn là ở Test Case đầu tiên, Test Case có số lượng mẫu vật liệu và sản phẩm cần cắt là ít nhất. Khi số lượng mẫu vật liệu và sản phẩm cần cắt tăng dần, ta cũng thấy tỉ lệ lãng phí có xu hướng giảm xuống.

Problem No.	No. stocks	No. products	Total area	Waste area	Waste rate	Average time (s)
1	36	30	9700	2922	0.3012	0.1209
2	49	50	40632	4699	0.1156	0.1516
3	64	60	42695	3255	0.0762	0.2026
4	81	80	33991	2655	0.0781	0.2125
5	100	100	41590	3433	0.0825	0.3340
6	100	110	72390	6533	0.0902	0.2690
7	100	120	119863	7599	0.0634	0.3263
8	100	130	116880	8187	0.0700	0.3254
9	100	140	70205	7157	0.1019	0.3559
10	100	150	138275	9083	0.0657	0.3796

Table 11: Kết quả đánh giá nội bộ của mô hình Reinforcement Learning.

- Cột "Average time" cho thấy thời gian chạy thuật toán là khá nhanh do mô hình mạng thần kinh tương đối đơn giản, có xu hướng tăng lên khi kích thước không gian nghiệm tăng.

Tóm lại, giải thuật Reinforcement Learning có phần nhỉnh hơn giải thuật tham lam kết hợp Annealing, có khả năng đưa ra quyết định hành động tương đối nhanh mà vẫn đảm bảo được độ chính xác của hành động. Yếu điểm của Reinforcement Learning so với các giải thuật khác chủ yếu đến từ dung lượng bộ nhớ cần sử dụng để lưu trữ mạng thần kinh và thời gian Train khá lâu, hơn nữa việc xây dựng ra một mô hình hợp lí cũng khá khó khăn.

5.5 Case Study: Ứng dụng 2D CSP trong ngành sản xuất bao bì

Để có cái nhìn khách quan hơn về hiệu suất của giải thuật tham lam kết hợp Simulated Annealing và mô hình Reinforcement Learning nhóm đã xây dựng, nhóm sẽ so sánh kết quả của chúng so với kết quả của phần mềm đã có trên thị trường là CutLogic 2D (phần mềm này sử dụng giải thuật di truyền nâng cao). Kết quả so sánh sẽ dựa trên Case Study sau:

Công ty X chuyên sản xuất các bao bì Carton phục vụ cho ngành dược phẩm và thực phẩm. Công ty đã nhận được một đơn đặt hàng từ một đối tác làm ăn lớn, yêu cầu sản xuất ra các hộp Carton với các kích thước tiêu chuẩn khác nhau. Tuy nhiên, nguồn nguyên liệu hiện tại chỉ có khoảng 100 tấm bìa Carton với kích thước không đều, do đã tái sử dụng lại các tấm bìa Carton cũ hoặc lấy từ các lô hàng dư thừa. Mục tiêu đặt ra là tìm cách để cắt các tấm bìa Carton này để tạo ra các sản phẩm đáp ứng yêu cầu mà tổng diện tích Carton bị thừa là ít nhất, cố gắng tận dụng tối đa nguồn nguyên liệu sẵn có.

Bảng bên dưới là danh sách kích thước của 100 tấm bìa Carton có sẵn (chiều rộng $\times$ chiều cao, đơn vị mm):

STT	Kích thước	STT	Kích thước	STT	Kích thước	STT	Kích thước	STT	Kích thước
1	547 × 565	21	691 × 915	41	1773 × 1747	61	2264 × 2160	81	3541 × 3038
2	583 × 698	22	923 × 1216	42	1442 × 1588	62	2043 × 1997	82	3233 × 3418
3	578 × 722	23	900 × 1067	43	1451 × 1425	63	2650 × 2288	83	3133 × 3223
4	551 × 681	24	1244 × 1123	44	1609 × 1965	64	2488 × 2453	84	3586 × 3233
5	683 × 581	25	1411 × 864	45	1545 × 1858	65	2734 × 2690	85	2751 × 3376
6	852 × 770	26	1149 × 1233	46	1564 × 1695	66	2681 × 2327	86	3044 × 3622
7	722 × 928	27	1408 × 1043	47	1632 × 1877	67	2670 × 2651	87	2961 × 3158
8	982 × 849	28	1127 × 1560	48	1940 × 2153	68	2408 × 2817	88	3460 × 3501
9	897 × 977	29	1143 × 1444	49	1974 × 2239	69	2497 × 2547	89	3138 × 3143
10	895 × 816	30	1366 × 1587	50	1770 × 1791	70	2995 × 2433	90	3897 × 3153
11	869 × 891	31	1703 × 1550	51	2287 × 2032	71	2884 × 2591	91	3919 × 3270
12	1114 × 1196	32	1481 × 1661	52	2304 × 2165	72	2484 × 2732	92	3727 × 3218
13	1199 × 1250	33	1593 × 1444	53	2098 × 1866	73	2727 × 2569	93	3872 × 3375
14	1190 × 1087	34	1640 × 1736	54	1963 × 2003	74	2649 × 2724	94	3246 × 4036
15	1343 × 1323	35	1425 × 1377	55	2077 × 2143	75	3170 × 3139	95	3372 × 3650
16	1146 × 1289	36	1518 × 1437	56	2022 × 2199	76	2789 × 2726	96	3790 × 3515
17	1181 × 1161	37	2007 × 1885	57	2153 × 2316	77	2721 × 2876	97	3423 × 4012
18	1354 × 1379	38	1946 × 1684	58	2162 × 2615	78	3178 × 2740	98	4010 × 3503
19	1450 × 1352	39	1816 × 1917	59	2631 × 2733	79	2923 × 3438	99	3658 × 4439
20	1329 × 1381	40	1765 × 1950	60	2742 × 2456	80	2920 × 3411	100	4443 × 3952

Table 12: Danh sách kích thước của 100 tấm bìa Carton.

Danh sách kích thước các sản phẩm cần cắt như sau (chiều rộng × chiều cao, đơn vị mm):

STT	Kích thước	Số lượng yêu cầu
1	150 × 100	20
2	150 × 120	18
3	200 × 100	18
4	250 × 300	16
5	800 × 1000	16
6	1000 × 1400	15
7	1300 × 1600	15
8	1500 × 1800	10
9	1800 × 2400	10
10	2000 × 2500	5

Table 13: Danh sách các sản phẩm cần cắt.

Bảng sau là cách cắt của phần mềm CutLogic 2D:

Index stock được cắt	Các sản phẩm được cắt bên trong	Diện tích thừa
98	$2 \times (2000 \times 2500)$, $1 \times (1000 \times 1400)$, $2 \times (800 \times 1000)$	1047030
97	$1 \times (1800 \times 2400)$, $1 \times (1500 \times 1800)$, $3 \times (1300 \times 1600)$, $5 \times (200 \times 100)$	373076
99	$1 \times (2000 \times 2500)$, $2 \times (1800 \times 2400)$, $2 \times (800 \times 1000)$, $7 \times (250 \times 300)$	472862
83	$2 \times (1500 \times 1800)$, $2 \times (1300 \times 1600)$, $11 \times (150 \times 120)$	339659
68	$1 \times (1800 \times 2400)$, $1 \times (1000 \times 1400)$, $1 \times (800 \times 1000)$	263336
27	$1 \times (1000 \times 1400)$	68544
92	$1 \times (1800 \times 2400)$, $2 \times (1300 \times 1600)$, $2 \times (1000 \times 1400)$, $2 \times (200 \times 100)$	673486
89	$2 \times (1500 \times 1800)$, $1 \times (1300 \times 1600)$, $1 \times (1000 \times 1400)$, $5 \times (250 \times 300)$, $2 \times (200 \times 100)$	567734
45	$1 \times (1500 \times 1800)$	170610
91	$1 \times (1800 \times 2400)$, $3 \times (1300 \times 1600)$, $1 \times (1000 \times 1400)$, $1 \times (200 \times 100)$	835130
86	$1 \times (2000 \times 2500)$, $2 \times (1000 \times 1400)$, $3 \times (800 \times 1000)$, $1 \times (200 \times 100)$, $2 \times (150 \times 120)$, $2 \times (150 \times 100)$	739368
88	$1 \times (1800 \times 2400)$, $1 \times (1300 \times 1600)$, $6 \times (800 \times 1000)$, $3 \times (150 \times 120)$, $2 \times (150 \times 100)$	829460
94	$1 \times (1800 \times 2400)$, $1 \times (1500 \times 1800)$, $1 \times (1300 \times 1600)$, $2 \times (1000 \times 1400)$, $1 \times (250 \times 300)$, $5 \times (200 \times 100)$	1025856
75	$2 \times (1500 \times 1800)$, $1 \times (1300 \times 1600)$, $1 \times (1000 \times 1400)$, $12 \times (150 \times 100)$	835130
70	$1 \times (1800 \times 2400)$, $1 \times (1000 \times 1400)$, $1 \times (800 \times 1000)$, $2 \times (200 \times 100)$, $2 \times (150 \times 120)$	690835
100	$1 \times (2000 \times 2500)$, $1 \times (1800 \times 2400)$, $1 \times (1500 \times 1800)$, $1 \times (1300 \times 1600)$, $1 \times (1000 \times 1400)$, $3 \times (250 \times 300)$	1833736
29	$1 \times (1000 \times 1400)$, $4 \times (150 \times 100)$	190492

Table 14: Cách cắt theo phần mềm CutLogic 2D.

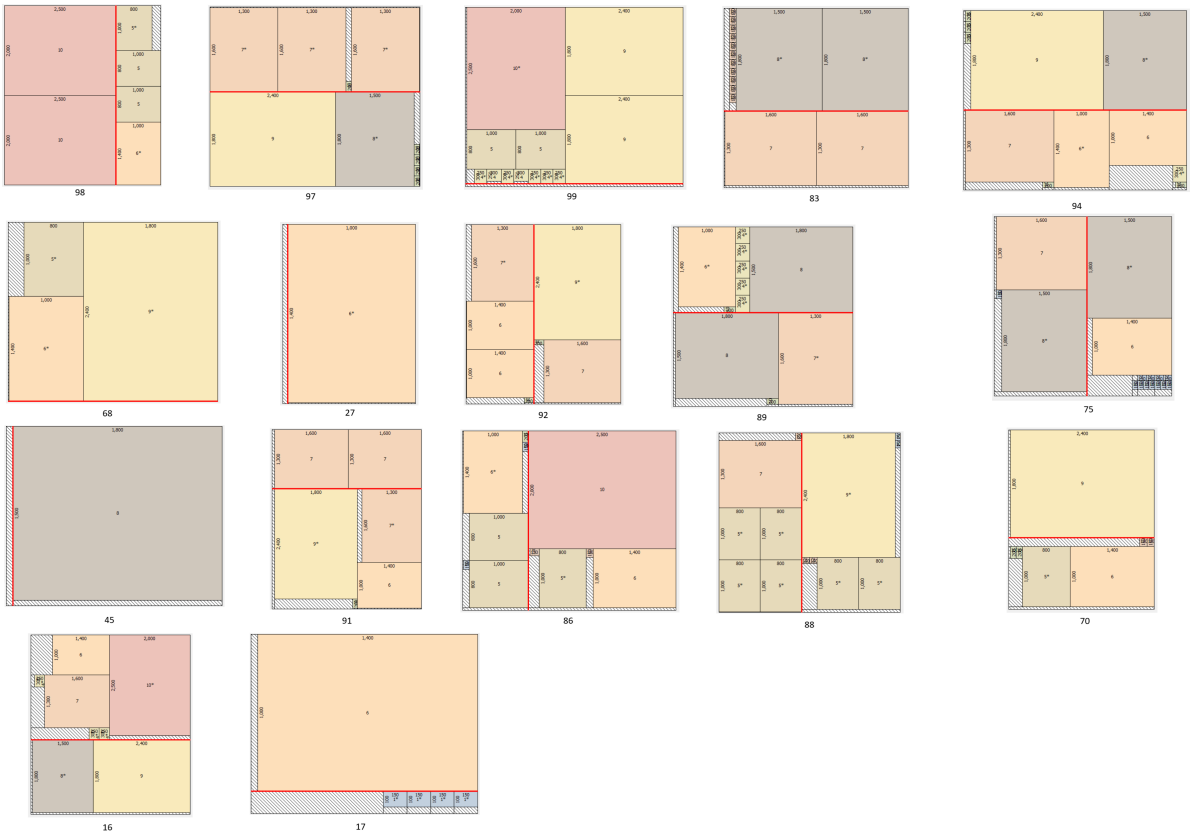


Figure 12: *Minh họa cách cắt các tấm bìa Carton bằng phần mềm CutLogic 2D.*

Bảng sau là cách cắt theo giải thuật tham lam kết hợp Simulated Annealing:

Index stock được cắt	Các sản phẩm được cắt bên trong	Diện tích thừa
80	$1 \times (1500 \times 1800)$, $3 \times (1300 \times 1600)$	1020120
81	$3 \times (1000 \times 1400)$, $4 \times (800 \times 1000)$	3357558
84	$2 \times (1300 \times 1600)$, $3 \times (1000 \times 1400)$, $1 \times (800 \times 1000)$	2433538
90	$2 \times (1500 \times 1800)$, $2 \times (1300 \times 1600)$, $1 \times (800 \times 1000)$	1927241
91	$2 \times (1800 \times 2400)$, $3 \times (800 \times 1000)$	1775130
92	$4 \times (1300 \times 1600)$, $2 \times (1000 \times 1400)$	873486
93	$2 \times (1800 \times 2400)$, $3 \times (800 \times 1000)$	2028000
94	$1 \times (1800 \times 2400)$, $1 \times (1500 \times 1800)$, $2 \times (1300 \times 1600)$, $1 \times (800 \times 1000)$	1120856
95	$1 \times (1800 \times 2400)$, $2 \times (1500 \times 1800)$, $1 \times (1000 \times 1400)$	1187800
96	$2 \times (1800 \times 2400)$, $2 \times (1000 \times 1400)$, $1 \times (800 \times 1000)$	1081850
97	$1 \times (1800 \times 2400)$, $3 \times (1500 \times 1800)$	1313076
98	$2 \times (2000 \times 2500)$, $2 \times (1000 \times 1400)$, $1 \times (800 \times 1000)$	447030
99	$1 \times (2000 \times 2500)$, $1 \times (1800 \times 2400)$, $1 \times (1500 \times 1800)$, $1 \times (1000 \times 1400)$, $1 \times (800 \times 1000)$, $8 \times (250 \times 300)$	1417862
100	$2 \times (2000 \times 2500)$, $2 \times (1300 \times 1600)$, $1 \times (1000 \times 1400)$, $8 \times (250 \times 300)$, $18 \times (200 \times 100)$, $18 \times (150 \times 120)$, $20 \times (150 \times 100)$	414736

Table 15: Cách cắt theo giải thuật tham lam kết hợp Simulated Annealing.

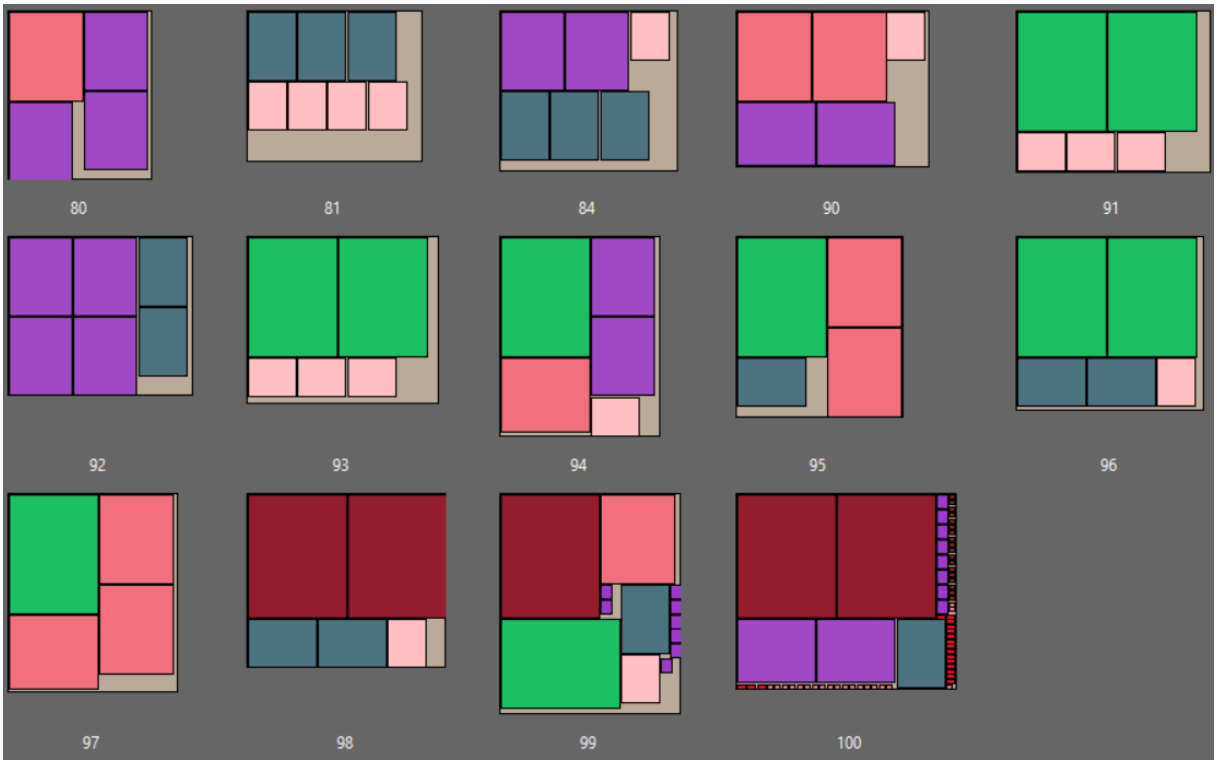


Figure 13: *Minh họa cách cắt các tấm bìa Carton theo giải thuật tham lam kết hợp Simulated Annealing (ảnh lấy từ phần mềm nhóm tạo).*

Bảng sau là cách cắt theo mô hình Reinforcement Learning của nhóm:

Index stock được cắt	Các sản phẩm được cắt bên trong	Diện tích thừa
23	$1 \times (800 \times 1000)$	160300
83	$1 \times (2000 \times 2500)$, $2 \times (1000 \times 1400)$, $16 \times (250 \times 300)$, $18 \times (200 \times 100)$, $18 \times (150 \times 120)$, $19 \times (150 \times 100)$	128659
84	$1 \times (1300 \times 1600)$, $3 \times (1000 \times 1400)$, $3 \times (800 \times 1000)$	2913538
86	$4 \times (1500 \times 1800)$	225368
88	$4 \times (1300 \times 1600)$, $3 \times (800 \times 1000)$	1393460
90	$1 \times (1500 \times 1800)$, $2 \times (1300 \times 1600)$, $2 \times (1000 \times 1400)$	2627241
91	$2 \times (1800 \times 2400)$, $3 \times (800 \times 1000)$	1775130
92	$4 \times (1300 \times 1600)$, $2 \times (1000 \times 1400)$	873486
93	$2 \times (1800 \times 2400)$, $3 \times (800 \times 1000)$	2028000
94	$1 \times (1800 \times 2400)$, $1 \times (1500 \times 1800)$, $2 \times (1300 \times 1600)$, $1 \times (800 \times 1000)$	1120856
96	$2 \times (1800 \times 2400)$, $2 \times (1000 \times 1400)$, $1 \times (800 \times 1000)$	1081850
97	$1 \times (1800 \times 2400)$, $3 \times (1500 \times 1800)$	1313076
98	$1 \times (2000 \times 2500)$, $1 \times (1800 \times 2400)$, $2 \times (1000 \times 1400)$	1927030
99	$1 \times (2000 \times 2500)$, $1 \times (1800 \times 2400)$, $1 \times (1500 \times 1800)$, $1 \times (1000 \times 1400)$, $1 \times (800 \times 1000)$	2017862
100	$2 \times (2000 \times 2500)$, $2 \times (1300 \times 1600)$, $1 \times (1000 \times 1400)$, $1 \times (150 \times 100)$	1120856

Table 16: Cách cắt theo mô hình Reinforcement Learning.

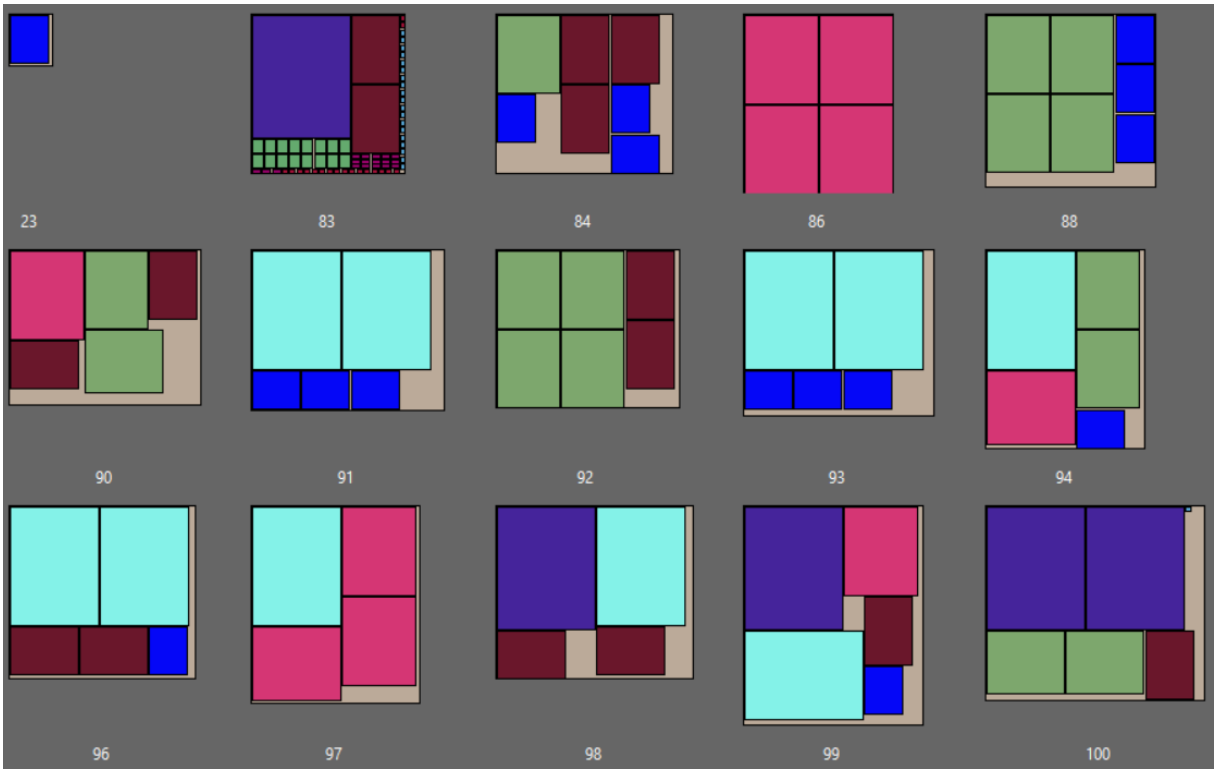


Figure 14: *Minh họa cách cắt các tấm bìa Carton theo mô hình Reinforcement Learning (ảnh lấy từ phần mềm nhóm tạo).*

Tóm lại, ta có bảng so sánh tổng quan như sau:

	Reinforcement Learning	Greedy with Simulated Annealing	CutLogic 2D
Tổng diện tích các tấm bìa đã cắt (mm^2)	183953592	182782283	172595844
Tổng diện tích thừa (mm^2)	21569592	20398283	10211844
Tỉ lệ lãng phí	0.1173	0.1116	0.0592
Thời gian chạy (s)	0.7462	9.2767	66

Table 17: So sánh hiệu suất của giải thuật tham lam kết hợp Simulated Annealing và mô hình Reinforcement Learning với phần mềm CutLogic 2D.

Như vậy, đã có sự đánh đổi giữa độ tối ưu và thời gian chạy ở đây, Reinforcement Learning có thời gian chạy gần như tức thời, nhưng lại có tỉ lệ lãng phí diện tích là lớn nhất, khoảng 11.73%. Trong khi đó, giải thuật tham lam kết hợp Simulated Annealing tối ưu hơn một chút, với tỉ lệ lãng phí giảm còn 11.16%, nhưng thời gian chạy lại lớn hơn nhiều lần. Và cuối cùng là phần mềm CutLogic 2D là tối ưu hơn hẳn, tỉ lệ lãng phí giảm gần một nửa so với 2 giải thuật trước, nhưng thời gian chạy lại lên tới 66 giây. Điều này cho thấy, không có một giải thuật nào là mạnh nhất, việc lựa chọn giải thuật sẽ phụ thuộc vào hoàn cảnh và điều kiện sản xuất.

6 Kết luận

6.1 Hướng dẫn sử dụng công cụ

Trong thư mục dự án, tại thư mục "2D CSP Solver", là mã nguồn của phần mềm nhóm đã tạo ra để giải bài toán 2D CSP, nhóm đã hiện thực 4 giải thuật là Simplex, Column Generation, tham lam kết hợp Simulated Annealing, và cuối cùng là Reinforcement Learning. Đối với Reinforcement Learning, trước khi chạy thì người dùng cần tải File weight tại đây và đặt vào trong thư mục "2D CSP Solver". Phần mềm này được viết bằng ngôn ngữ Python, nên người dùng cần cài đặt Python trước khi chạy. Lệnh cài đặt các thư viện cần thiết và lệnh chạy như sau:

```
pip install -r requirements.txt
python main.py
```

Giao diện ban đầu của phần mềm như sau:

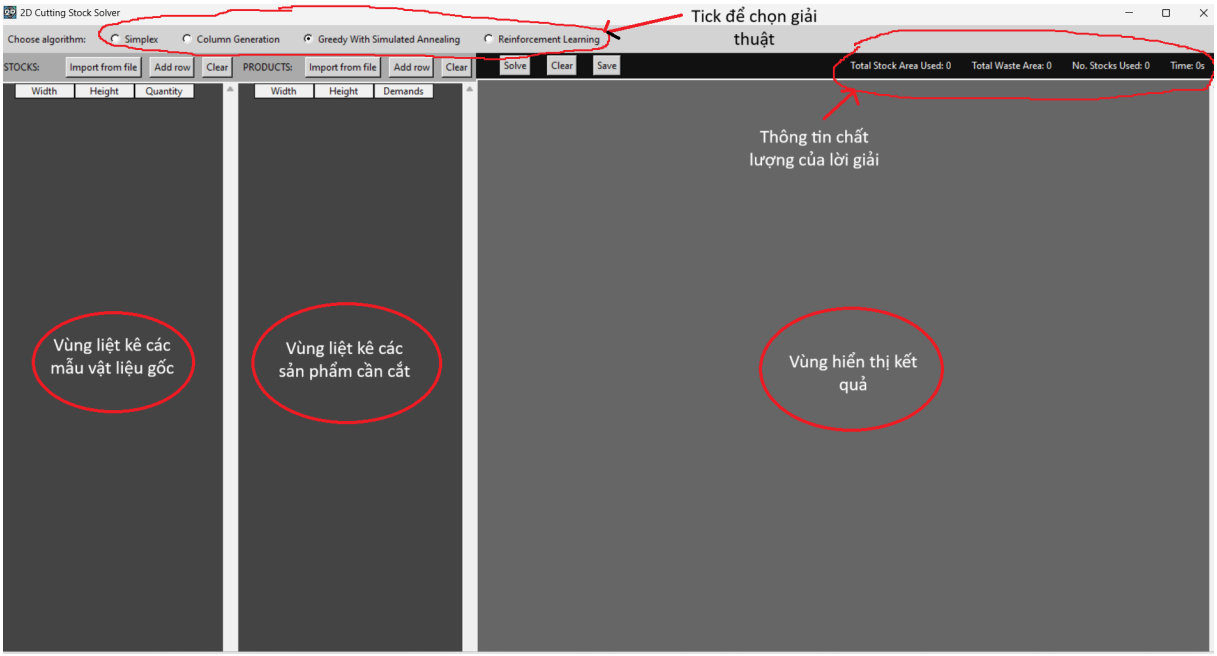


Figure 15: *Giao diện phần mềm.*

Tại Stocks Panel:

- 3 cột "Width", "Height" và "Quantity" lần lượt là chiều rộng, chiều cao và số lượng tối đa của các mẫu vật liệu. Click nút "Add row" để thêm 1 mẫu vật liệu mới và nhập chiều rộng, chiều cao và số lượng tối đa của nó vào các ô Input (có thể nhập inf vào cột "Quantity" ứng với sử dụng không giới hạn).
- Click nút "Clear" để xóa tất cả các mẫu vật liệu.
- Click nút "Import from file" và chọn 1 File dữ liệu để Import vào Stocks Panel, File dữ liệu này mỗi dòng sẽ ứng với một mẫu vật liệu, có cú pháp <Chiều rộng>,<Chiều cao>,<Số lượng tối đa>.

Tại Products Panel:

- 3 cột "Width", "Height" và "Quantity" lần lượt là chiều rộng, chiều cao và số lượng yêu cầu của các sản phẩm. Click nút "Add row" để thêm 1 sản phẩm mới và nhập chiều rộng, chiều cao và số lượng yêu cầu của nó vào các ô Input.
- Click nút "Clear" để xóa tất cả các sản phẩm.
- Click nút "Import from file" và chọn 1 File dữ liệu để Import vào Products Panel, File dữ liệu này mỗi dòng sẽ ứng với một sản phẩm, có cú pháp <Chiều rộng>,<Chiều cao>,<Số lượng yêu cầu>.

Để tiến hành giải bài toán, đầu tiên Tick chọn 1 trong 4 giải thuật được liệt kê ở thanh trên cùng, sau đó Click nút "Solve". Click nút "Clear" để xóa kết quả. Click nút "Save" để lưu kết quả vào 1 File.

6.2 Kết quả làm việc

Trong quá trình làm bài tập lớn, nhóm đã hoàn thành được các nhiệm vụ sau:

- Giới thiệu được một cách tổng quan về bài toán 2D CSP, về lịch sử hình thành và vai trò của nó trong các ngành công nghiệp.
- Đã mô hình hóa được bài toán ở nhiều trường hợp khác nhau dưới dạng quy hoạch tuyến tính nguyên.
- Đã đề xuất được một số giải thuật dùng để giải quyết bài toán, và phân tích được điểm mạnh, yếu của chúng.
- Đã đưa ra được một Case Study liên quan đến 2D CSP và các giải pháp để giải quyết nó.
- Đã phát triển được một phần mềm để giải bài toán.
- Hoàn thành việc so sánh hiệu suất giữa các giải thuật với nhau và với các phần mềm khác dựa trên các bộ dữ liệu thực tế.

6.3 Các thiếu sót

Mặc dù đã rất nỗ lực, nhưng bài tập lớn của nhóm vẫn còn tương đối nhiều thiếu sót, kể đến như:

- Các giải thuật đề xuất vẫn chưa thực sự tối ưu, minh chứng là tỉ lệ lãng phí của chúng gần gấp đôi so với khi giải bằng phần mềm trên thị trường.
- Chưa thực sự đưa ra một mô hình Reinforcement Learning phức tạp để giải bài toán.

6.4 Hướng phát triển trong tương lai

Rút kinh nghiệm từ các thiết sót, trong tương lai, nhóm sẽ dự định tiếp tục phát triển phần mềm trên các khía cạnh sau:

- Nghiên cứu và hiện thực được một mô hình Reinforcement Learning phức tạp và tối ưu hơn.
- Tìm cách giải bài toán trong trường hợp 3 chiều.